

EXPERIMENT:1-15

NAME - SIDDHANT RAI

BATCH - 77

SAPID - 590029215

Experiment 1: Installation, Environment Setup and Starting with C Language

1. Write a C program to print "Hello World"

- ALGORITHM:

- 1: Start the program
- 2: Display the message **"Hello World"** on the screen
- 3: Return 0 to indicate successful execution
- 4: Stop the program

PSUEDOCODE:

```
START  
    PRINT "Hello World"  
    RETURN 0  
END
```

INPUT:

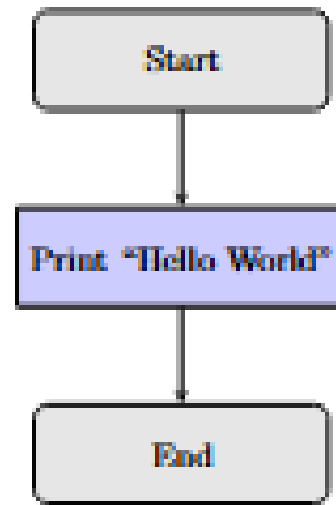
```
1  #include <stdio.h>
2  int main() {
3      printf("Hello World\n");
4      return 0;
5  }
```

OUTPUT:

```
Hello World
```

```
=== Code Execution Successful ===
```

FLOWCHART:



2. Write a C program to print the address in multiple lines (new line)

- **Algorithm**

1. Start the program.
2. Print the first line of the address: *"123 Example Street"*.
3. Print the second line: *"City Name"*.
4. Print the third line: *"State Name"*.
5. Print the fourth line: *"Country"*.
6. End the program.

PSUEDOCODE:

```
BEGIN  
    PRINT "123 Example Street"  
    PRINT "City Name"  
    PRINT "State Name"  
    PRINT "Country"  
END
```


INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int n;
4      printf("Enter how many times to print the address: ");
5      scanf("%d", &n);
6      for(int i = 0; i < n; i++) {
7          printf("123, Green Park\n");
8          printf("New Delhi\n");
9          printf("India\n\n");
10     }
11     return 0;
12 }
13
```

OUTPUT:

```
Enter how many times to print the address: 5
123, Green Park
New Delhi
India

123, Green Park
New Delhi
India

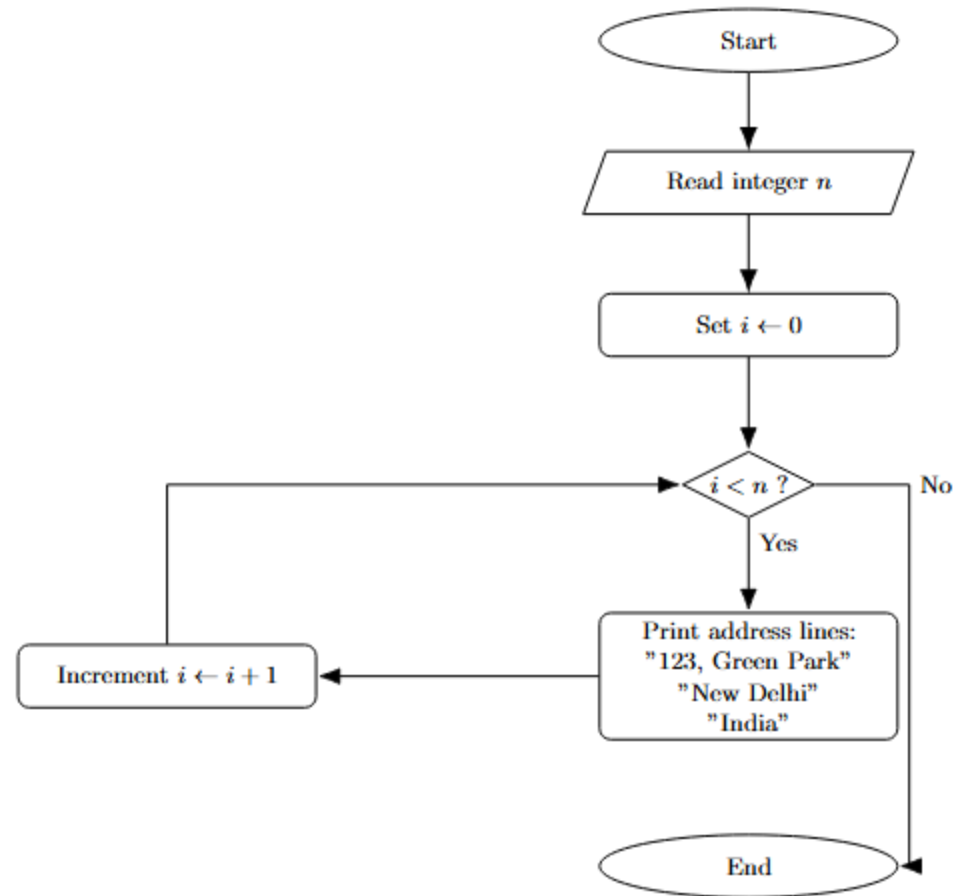
123, Green Park
New Delhi
India

123, Green Park
New Delhi
India

123, Green Park
New Delhi
India

=== Code Execution Successful ===
```

FLOWCHART:



3. Write a program that prompts the user to enter their name and age

Algorithm

1. Start
2. Declare a character array `name` and an integer variable `age`
3. Display: *"Enter your name:"*
4. Read the value into `name`
5. Display: *"Enter your age:"*
6. Read the value into `age`
7. Display the entered name and age
8. Stop

PSUEDOCODE:

```
START

DECLARE name as string (size 50)
DECLARE age as integer

PRINT "Enter your name: "
READ name

PRINT "Enter your age: "
READ age

PRINT "Name: " + name
PRINT "Age: " + age

END
```

INPUT:

```
1  #include <stdio.h>
2  int main() {
3      char name[50];
4      int age;
5      printf("Enter your name: ");
6      fgets(name, sizeof(name), stdin);
7      printf("Enter your age: ");
8      scanf("%d", &age);
9      printf("\n--- User Details ---\n");
10     printf("Name: %s", name);
11     printf("Age: %d\n", age);
12     return 0;
13 }
```

OUTPUT:

```
Enter your name: Siddhant  
Enter your age: 18
```

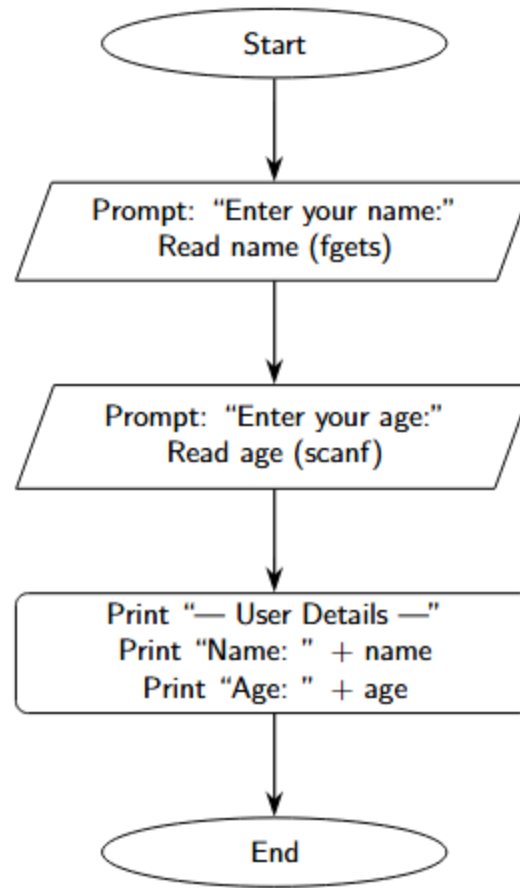
```
--- User Details ---
```

```
Name: Siddhant
```

```
Age: 18
```

```
--- Code Execution Successful ---
```

FLOWCHART:



4. Write a C program to add two numbers, take numbers from the user

Algorithm

1. Start
2. Declare three integer variables: a, b, and sum
3. Display "Enter first number:"
4. Read the value of a
5. Display "Enter second number:"
6. Read the value of b
7. Compute $\text{sum} = a + b$
8. Display the sum
9. End

PSUEDOCODE:

```
START
DECLARE a, b, sum as integers

DISPLAY "Enter first number: "
READ a

DISPLAY "Enter second number: "
READ b

SET sum = a + b

DISPLAY "Sum = " + sum

END
```

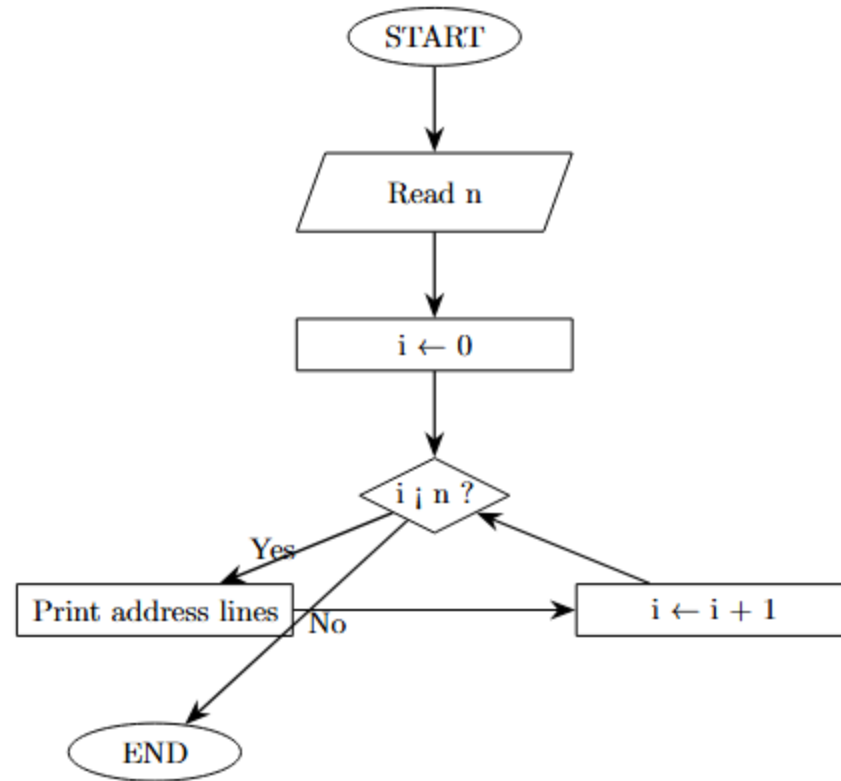
INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int a, b, sum;
4      printf("Enter first number: ");
5      scanf("%d", &a);
6      printf("Enter second number: ");
7      scanf("%d", &b);
8      sum = a + b;
9      printf("Sum = %d\n", sum);
10     return 0;
11 }
```

OUTPUT:

```
Enter first number: 32  
Enter second number: 43  
Sum = 75  
  
=== Code Execution Successful ===
```

FLOWCHART:



EXPERIMENT 2:Operator

1. WAP a C program to calculate the area and perimeter of a rectangle based on its length and width.

ALGORITHM:

1. Start
2. Declare variables: length, width, area, perimeter
3. Read length
4. Read width
5. Compute $\text{area} = \text{length} \times \text{width}$
6. Compute $\text{perimeter} = 2 \times (\text{length} + \text{width})$
7. Display area
8. Display perimeter
9. End

PSUEDOCODE:

```
START

DECLARE length, width, area, perimeter

PRINT "Enter length:"
READ length

PRINT "Enter width:"
READ width

area ← length * width
perimeter ← 2 * (length + width)

PRINT "Area =", area
PRINT "Perimeter =", perimeter

END
```

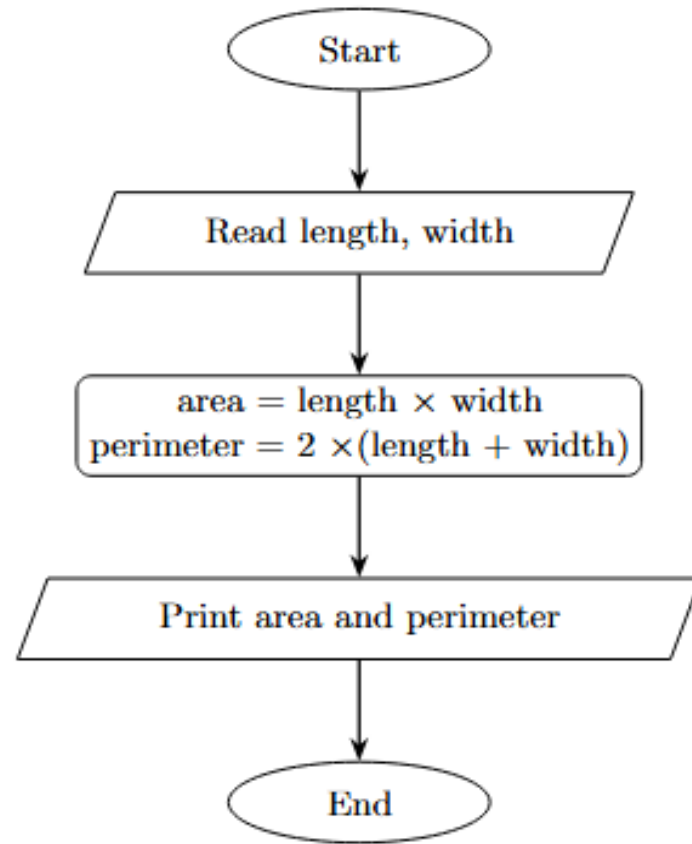
INPUT:

```
1  #include <stdio.h>
2  int main() {
3      float length, width, area, perimeter;
4      printf("Enter length of the rectangle: ");
5      scanf("%f", &length);
6      printf("Enter width of the rectangle: ");
7      scanf("%f", &width);
8      area = length * width;
9      perimeter = 2 * (length + width);
10     printf("Area of the rectangle = %.2f\n", area);
11     printf("Perimeter of the rectangle = %.2f\n", perimeter);
12     return 0;
13 }
14
```


OUTPUT:

```
Enter length of the rectangle: 12  
Enter width of the rectangle: 14  
Area of the rectangle = 168.00  
Perimeter of the rectangle = 52.00  
  
=== Code Execution Successful ===
```

FLOWCHART:



2. WAP a C program to convert temperature from Celsius to Fahrenheit using the formula: $F = (C * 9/5) + 32$.

ALGORITHM:

1. Start
2. Declare variables: `celsius`, `fahrenheit`
3. Display: "Enter temperature in Celsius:"
4. Read value into `celsius`
5. Compute Fahrenheit using formula:
6. $\text{fahrenheit} = (\text{celsius} \times 9/5) + 32$
 $\text{fahrenheit} = (\text{celsius} \times 9/5) + 32$
7. Display the Fahrenheit value
8. Stop

PSUEDOCODE:

```
START

DECLARE celsius, fahrenheit AS FLOAT

PRINT "Enter temperature in Celsius:"
INPUT celsius

fahrenheit ← (celsius * 9 / 5) + 32

PRINT "Temperature in Fahrenheit =", fahrenheit

END
```

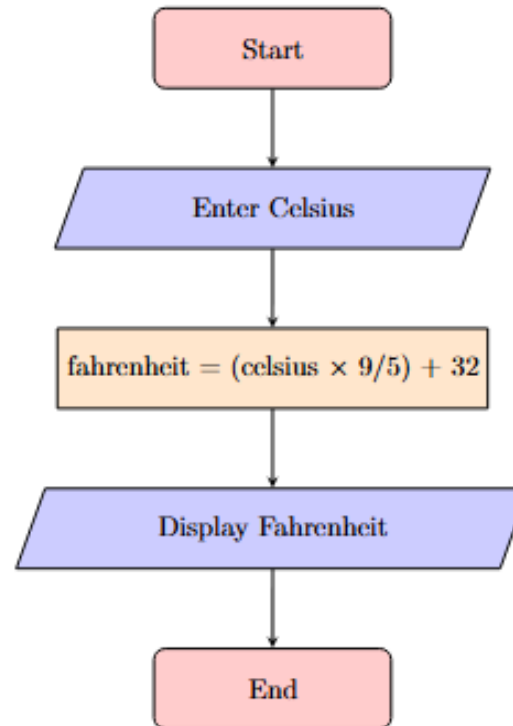
INPUT:

```
1  #include <stdio.h>
2  int main() {
3      float celsius, fahrenheit;
4      printf("Enter temperature in Celsius: ");
5      scanf("%f", &celsius);
6      fahrenheit = (celsius * 9.0 / 5.0) + 32;
7      printf("Temperature in Fahrenheit = %.2f\n", fahrenheit);
8      return 0;
9  }
```

OUTPUT:

```
Enter temperature in Celsius: 32  
Temperature in Fahrenheit = 89.60  
  
=== Code Execution Successful ===
```

FLOWCHART:



ACTIVITY:3.1

(Experiment 3.1: Write a program to check whether the given triangle is equilateral, isosceles or scalene.)

- **ALGORITHM:**

- **1.Start**

- **2.Input** three integers a, b, and c (the sides of the triangle).

- **3.Check triangle validity** using the triangle inequality condition:

- If $(a + b > c)$ AND $(a + c > b)$ AND $(b + c > a)$, then the triangle is valid.

- Else, print "The entered sides do not form a valid triangle." and **stop**.

- **4.If valid:**

- If $a == b$ && $b == c$, print "Equilateral triangle".

- Else if $(a == b)$ OR $(b == c)$ OR $(c == a)$, print "Isosceles triangle".

- Else, print "Scalene triangle".

- **5.End**

PSEUDOCODE:

```
START
DECLARE a, b, c

PRINT "Enter the three sides of triangle: "
READ a, b, c

IF (a + b > c) AND (a + c > b) AND (b + c > a) THEN
    IF (a == b) AND (b == c) THEN
        PRINT "Equilateral triangle"
    ELSE IF (a == b) OR (b == c) OR (c == a) THEN
        PRINT "Isosceles triangle"
    ELSE
        PRINT "Scalene triangle"
    ENDIF
ELSE
    PRINT "The entered sides do not form a valid triangle."
ENDIF

END
```

CODE-INPUT

```
#include <stdio.h>
int main() {
    int a, b, c;
    printf("Enter the three sides of triangle: ");
    scanf("%d %d %d", &a, &b, &c);
    if (a + b > c && a + c > b && b + c > a) {
        if (a == b && b == c) {
            printf("Equilateral triangle\n");
        } else if (a == b || b == c || c == a) {
            printf("Isosceles triangle\n");
        } else {
            printf("Scalene triangle\n");
        }
    } else {
        printf("The entered sides do not form a valid triangle.\n");
    }
    return 0;
}
```

CODE-OUTPUT

```
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> .\a.exe
Enter the three sides of triangle: 5 5 5
Equilateral triangle
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> gcc 3.1Triangle.c
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> .\a.exe
Enter the three sides of triangle: 5 5 7
Isosceles triangle
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> gcc 3.1Triangle.c
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> .\a.exe
Enter the three sides of triangle: 5 6 7
Scalene triangle
```

FLOWCHART



ACTIVITY:3.2

(Write a program to calculate BMI and display the category.)

ALGORITHM:

1.Start

2.Input weight in kilograms.

3.Input height in meters.

4.Compute BMI using the formula:

5. $BMI = \frac{\text{weight}}{\text{height} \times \text{height}}$ BMI=height*heightweight

6.Display the calculated BMI (rounded to two decimal places).

7.Classify BMI into categories:

If BMI < 15, print "Category: Starvation".

Else if 15.1 <= BMI <= 17.5, print "Category: Anorexic".

Else if 17.6 <= BMI <= 18.5, print "Category: Underweight".

Else if 18.6 <= BMI <= 24.9, print "Category: Ideal".

Else if 25 <= BMI <= 29.9, print "Category: Overweight".

Else if 30 <= BMI <= 39.9, print "Category: Obese".

Else if BMI >= 40, print "Category: Morbidly Obese".

Else, print "Category: Undefined Range".

8.End

PSUEDOCODE:

```
START
DECLARE weight, height, bmi (float)

PRINT "Enter weight in kg: "
READ weight

PRINT "Enter height in meters: "
READ height

bmi ← weight / (height * height)

PRINT "Your BMI is: ", bmi (formatted to 2 decimals)

IF bmi < 15 THEN
    PRINT "Category: Starvation"
ELSE IF bmi >= 15.1 AND bmi <= 17.5 THEN
    PRINT "Category: Anorexic"
ELSE IF bmi >= 17.6 AND bmi <= 18.5 THEN
    PRINT "Category: Underweight"
ELSE IF bmi >= 18.6 AND bmi <= 24.9 THEN
    PRINT "Category: Ideal"
ELSE IF bmi >= 25 AND bmi <= 29.9 THEN
    PRINT "Category: Overweight"
ELSE IF bmi >= 30 AND bmi <= 39.9 THEN
    PRINT "Category: Obese"
ELSE IF bmi >= 40 THEN
    PRINT "Category: Morbidly Obese"
ELSE
    PRINT "Category: Undefined Range"
ENDIF

END
```

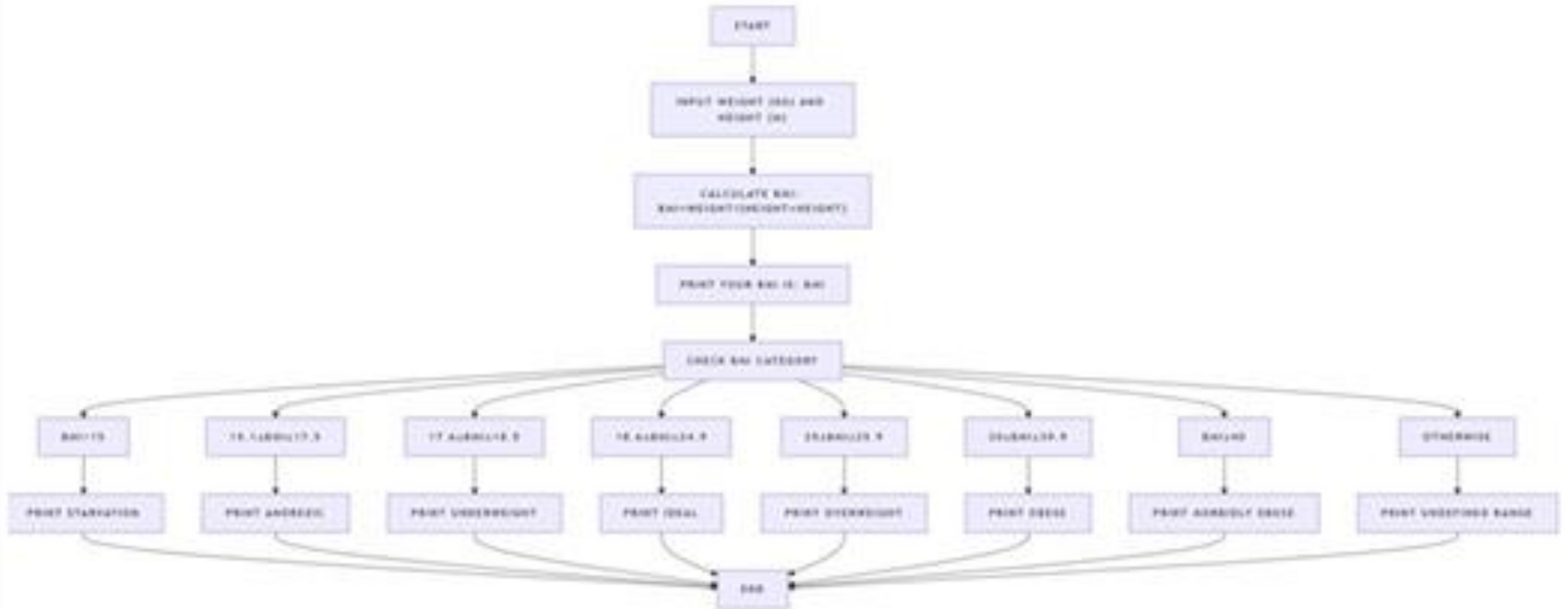
CODE-INPUT

```
Run Under C:\data.h>
int main() {
    float weight, height, bmi;
    printf("Enter weight in kg: ");
    scanf("%f", &weight);
    printf("Enter height in metres: ");
    scanf("%f", &height);
    bmi = weight / (height * height);
    printf("Your BMI is: %.2f\n", bmi);
    if (bmi < 15) {
        printf("Category: Starvation\n");
    } else if (bmi >= 15.1 && bmi <= 17.5) {
        printf("Category: Anorexic\n");
    } else if (bmi >= 17.6 && bmi <= 18.5) {
        printf("Category: Underweight\n");
    } else if (bmi >= 18.6 && bmi <= 24.9) {
        printf("Category: Ideal\n");
    } else if (bmi >= 25 && bmi <= 25.9) {
        printf("Category: Overweight\n");
    } else if (bmi >= 26 && bmi <= 29.9) {
        printf("Category: Obese\n");
    } else if (bmi >= 30) {
        printf("Category: Morbidly Obese\n");
    } else {
        printf("Category: Undefined Range\n");
    }
    return 0;
}
```

CODE-OUTPUT

```
Your BMI is: 3.75  
Category: Starvation  
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> gcc 3.2BMI.c  
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> .\a.exe  
Enter weight in kg: 100  
Enter height in meters: 2  
Your BMI is: 25.00  
Category: Overweight  
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> █
```


FLOWCHART



ACTIVITY:3.3

(Check if three points are collinear)

- **Algorithm:**

- **1.Start**

- **2.Input** the coordinates of three points:

Point A $\rightarrow (x_1, y_1)$

Point B $\rightarrow (x_2, y_2)$

Point C $\rightarrow (x_3, y_3)$

- **3.Compute area of triangle** formed by the three points using the determinant formula:

- $$\text{area} = \frac{1}{2} \times (x_1 \times (y_2 - y_3) + x_2 \times (y_3 - y_1) + x_3 \times (y_1 - y_2))$$

- **4.Check condition:**

If $\text{area} == 0 \rightarrow \text{Print "The points are collinear."}$

Else $\rightarrow \text{Print "The points are NOT collinear."}$

- **5.End**

PSEUDOCODE:

```
START
DECLARE x1, y1, x2, y2, x3, y3, area (float)

PRINT "Enter coordinates of first point (x1 y1): "
READ x1, y1

PRINT "Enter coordinates of second point (x2 y2): "
READ x2, y2

PRINT "Enter coordinates of third point (x3 y3): "
READ x3, y3

area = (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2)) / 2.0

IF area == 0 THEN
    PRINT "The points are collinear."
ELSE
    PRINT "The points are NOT collinear."
ENDIF

END
```

CODE-INPUT

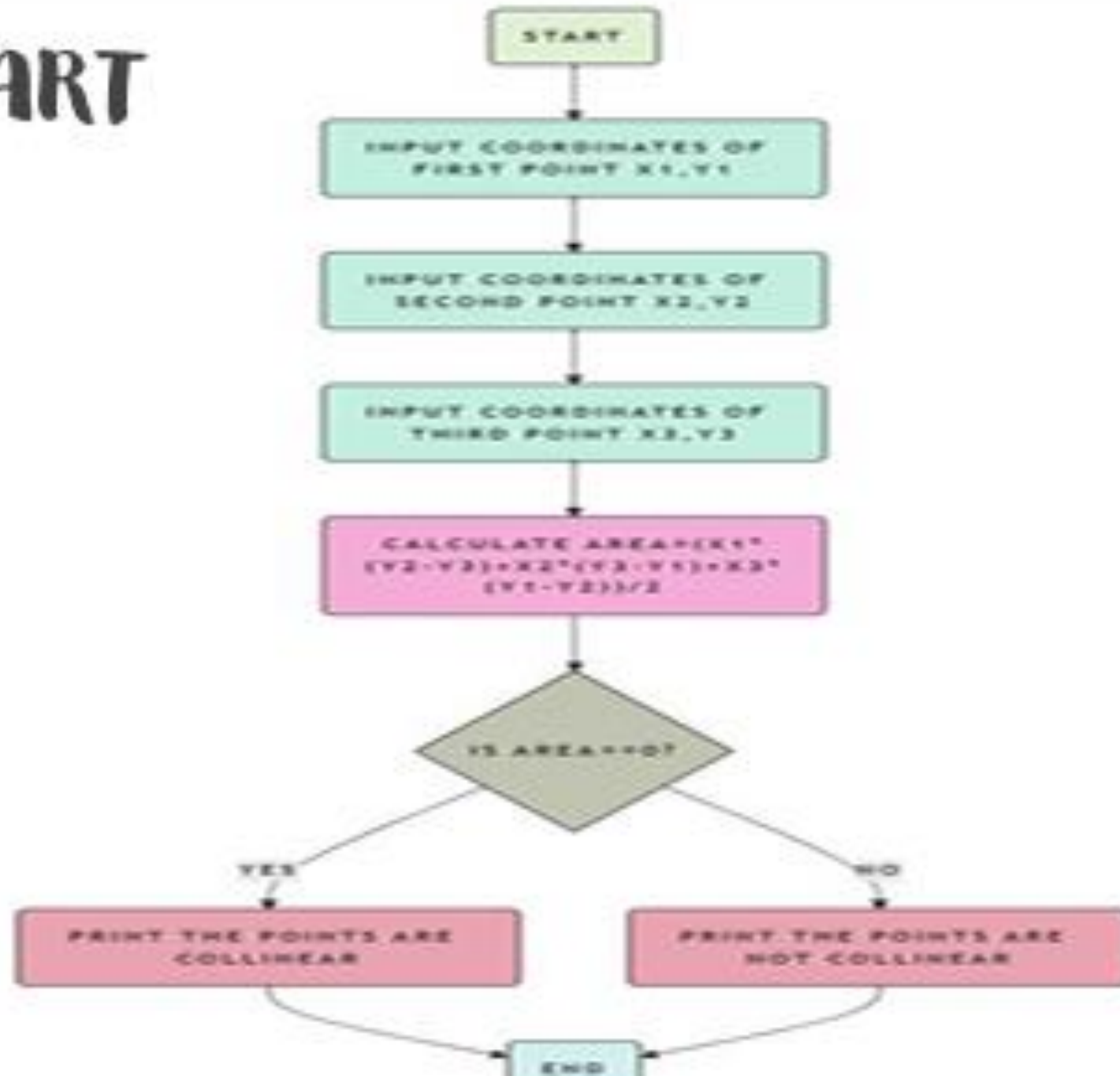
```
#include <stdio.h>
int main() {
    float x1, y1, x2, y2, x3, y3;
    printf("Enter coordinates of first point (x1 y1): ");
    scanf("%f %f", &x1, &y1);
    printf("Enter coordinates of second point (x2 y2): ");
    scanf("%f %f", &x2, &y2);
    printf("Enter coordinates of third point (x3 y3): ");
    scanf("%f %f", &x3, &y3);
    float area = (x1*(y2 - y3) + x2*(y3 - y1) + x3*(y1 - y2)) / 2.0;
    if (area == 0)
        printf("The points are collinear.\n");
    else
        printf("The points are NOT collinear.\n");
    return 0;
}
```



CODE-OUTPUT

```
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> gcc 3.3Collinear.c
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> .\a.exe
Enter coordinates of first point (x1 y1): 5 5
Enter coordinates of second point (x2 y2): 5 5
Enter coordinates of third point (x3 y3): 5 5
The points are collinear.
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> gcc 3.3Collinear.c
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> .\a.exe
Enter coordinates of first point (x1 y1): 5 6
Enter coordinates of second point (x2 y2): 6 7
Enter coordinates of third point (x3 y3): 6 6
The points are NOT collinear.
PS C:\Users\SIDDHANT RAI\OneDrive\Desktop\vscode> █
```

FLOWCHART



Activity 3.4

(to find the day of the week on 1st January of a given year)

- algorithm
- 1. Start
- Input year.
- Set days = 0.
- For each year y from 1900 to year - 1:
 - If y is a leap year (i.e., if y is odd or if y is 4 but not 100 or if y is 400), then add 366 to days.
 - Else add 365 to days.
- Compute day = days % 7.
- Map day to 1 for weekdays:
 - 0 → Monday
 - 1 → Tuesday
 - 2 → Wednesday
 - 3 → Thursday
 - 4 → Friday
 - 5 → Saturday
 - 6 → Sunday
- Print the weekday for 1st January of that year.
- End

PSUEDOCODE:

```
FUNCTION isLeap(year):
    IF (year % 400 == 0) OR ((year % 4 == 0) AND (year % 100 != 0)) THEN
        RETURN 1
    ELSE
        RETURN 0
    ENDIF
END FUNCTION

START
DECLARE year, y, day, days
days ← 0

PRINT "Enter year: "
READ year

FOR y FROM 1900 TO year - 1 DO
    IF isLeap(y) == 1 THEN
        days ← days + 366
    ELSE
        days ← days + 365
    ENDIF
END FOR

day ← days MOD 7

PRINT "1st January ", year, " was "

SWITCH day
CASE 0: PRINT "Monday"
CASE 1: PRINT "Tuesday"
CASE 2: PRINT "Wednesday"
CASE 3: PRINT "Thursday"
CASE 4: PRINT "Friday"
CASE 5: PRINT "Saturday"
CASE 6: PRINT "Sunday"
END SWITCH

END
```


CODE-INPUT

```
#include <stdio.h>
int is_leap(int y) {
    return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
}
int main(void) {
    int year, y;
    long days = 0;
    const int base = 1900;
    const char *dayname[7] = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"};
    if (scanf("%d", &year) != 1) return 0;
    if (year >= base) {
        for (y = base; y < year; ++y) days += 365 + is_leap(y);
    } else {
        for (y = year; y < base; ++y) days -= 365 + is_leap(y);
    }
    int idx = (int)((days % 7 + 7) % 7);
    printf("1-January-%d is %s\n", year, dayname[idx]);
    return 0;
}
```

CODE-OUTPUT

```
#include <stdio.h>
int isLeap(int year) {
    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
        return 1;
    return 0;
}
int main() {
    int year, days = 0;
    printf("Enter year: ");
    scanf("%d", &year);
    for (int y = 1900; y < year; y++) {
        if (isLeap(y))
            days += 366;
        else
            days += 365;
    }
    int day = days % 7;
    printf("1st January %d was ", year);
    switch (day) {
        case 0: printf("Monday\n"); break;
        case 1: printf("Tuesday\n"); break;
        case 2: printf("Wednesday\n"); break;
        case 3: printf("Thursday\n"); break;
        case 4: printf("Friday\n"); break;
        case 5: printf("Saturday\n"); break;
        case 6: printf("Sunday\n"); break;
    }
    return 0;
}
```

FLOWCHART



Activity-5:

C program to find the rectangle with the highest perimeter among three rectangles using ternary operator

- ALGORITHM
- **Start**
- **Input** length (l1) and breadth (b1) of rectangle 1.
- **Input** length (l2) and breadth (b2) of rectangle 2.
- **Input** length (l3) and breadth (b3) of rectangle 3.
- **Calculate** perimeter of rectangle 1:
 $p1 = 2 \times (l1 + b1)$
- **Calculate** perimeter of rectangle 2:
 $p2 = 2 \times (l2 + b2)$
- **Calculate** perimeter of rectangle 3:
 $p3 = 2 \times (l3 + b3)$
- **Find maximum perimeter** among p1, p2, p3.
- If p1 is the largest $\rightarrow \max = p1$
- Else if p2 is the largest $\rightarrow \max = p2$
- Else $\rightarrow \max = p3$
- **Display** which rectangle has the highest perimeter and its value.
- **Stop**

PSUEDOCODE:

```
BEGIN

DECLARE l1, b1, l2, b2, l3, b3, p1, p2, p3, max

PRINT "Enter length and breadth of rectangle 1"
READ l1, b1

PRINT "Enter length and breadth of rectangle 2"
READ l2, b2

PRINT "Enter length and breadth of rectangle 3"
READ l3, b3

p1 ← 2 × (l1 + b1)
p2 ← 2 × (l2 + b2)
p3 ← 2 × (l3 + b3)

IF p1 > p2 THEN
  IF p1 > p3 THEN
    max ← p1
  ELSE
    max ← p3
  ENDIF
ELSE
  IF p2 > p3 THEN
    max ← p2
  ELSE
    max ← p3
  ENDIF
ENDIF

IF max = p1 THEN
  PRINT "Rectangle 1 has the highest perimeter:", p1
ELSE IF max = p2 THEN
  PRINT "Rectangle 2 has the highest perimeter:", p2
ELSE
  PRINT "Rectangle 3 has the highest perimeter:", p3
ENDIF

END
```

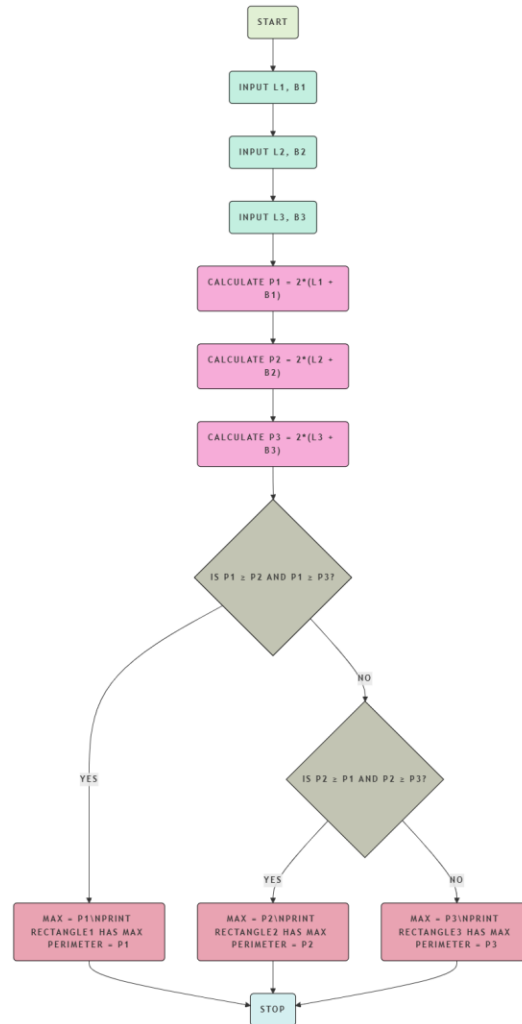
INPUT

```
C 3.1.5.c > ...
1  #include <stdio.h>
2  int main() {
3      int l1, b1, l2, b2, l3, b3;
4      int p1, p2, p3, max;
5      printf("Enter length and breadth of rectangle 1: ");
6      scanf("%d %d", &l1, &b1);
7      printf("Enter length and breadth of rectangle 2: ");
8      scanf("%d %d", &l2, &b2);
9      printf("Enter length and breadth of rectangle 3: ");
10     scanf("%d %d", &l3, &b3);
11     p1 = 2 * (l1 + b1);
12     p2 = 2 * (l2 + b2);
13     p3 = 2 * (l3 + b3);
14     max = (p1 > p2) ? ((p1 > p3) ? p1 : p3) : ((p2 > p3) ? p2 : p3);
15     if (max == p1)
16         printf("Rectangle 1 has the highest perimeter: %d\n", p1);
17     else if (max == p2)
18         printf("Rectangle 2 has the highest perimeter: %d\n", p2);
19     else
20         printf("Rectangle 3 has the highest perimeter: %d\n", p3);
21     return 0;
22 }
23
```

OUTPUT

```
cd "/Users/nitishkumar/Documents/Code/pracricecode/" && gcc 3.1.5.c -o 3.1.5 && "/Users/nitishkumar/
Documents/Code/pracricecode/"3.1.5
NitishK@Nitishs-MacBook-Air Code % cd "/Users/nitishkumar/Documents/Code/pracricecode/" && gcc 3.1.5
.c -o 3.1.5 && "/Users/nitishkumar/
Documents/Code/pracricecode/"3.1.5
Enter length and breadth of rectangle 1: cd "/Users/nitishkumar/Documents/Code/pracricecode/" && gcc
3.1.5.c -o 3.1.5 && "/Users/nitishkumar/Documents/Code/pracricecode/"3.1.5
Enter length and breadth of rectangle 2: Enter length and breadth of rectangle 3: Rectangle 3 has th
e highest perimeter: -95068690
NitishK@Nitishs-MacBook-Air pracricecode %
```

FLOWCHART



Experiment-3.2

Activity-1:

a program that reads a series of numbers from the user and counts how many of them are positive, negative, and zero. The program should continue to prompt the user for numbers until they choose to stop. After the user decides to stop entering numbers, the program should display the counts of positive numbers, negative numbers, and zeroes entered.

ALGORITHM

Start

Initialize three counters:

```
countPositive = 0
```

```
countNegative = 0
```

```
countZero = 0
```

Repeat the following steps **at least once**:

- Input a number (num).
- If $\text{num} > 0$, increment countPositive.
- Else if $\text{num} < 0$, increment countNegative.
- Else, increment countZero.
- Ask user: "Do you want to enter more numbers? (Y/N)".

Continue the loop while the user enters Y or y.

After loop ends, display the results:

Print number of positive entries.

Print number of negative entries.

Print number of zeros entered.

Stop

PSUEDOCODE:

```
BEGIN

    countPositive ← 0
    countNegative ← 0
    countZero ← 0

    REPEAT
        PRINT "Enter a number: "
        READ num

        IF num > 0 THEN
            countPositive ← countPositive + 1
        ELSE IF num < 0 THEN
            countNegative ← countNegative + 1
        ELSE
            countZero ← countZero + 1
        ENDIF

        PRINT "Do you want to enter more numbers? (Y/N): "
        READ choice

    UNTIL choice ≠ 'Y' AND choice ≠ 'y'

    PRINT "Summary of numbers entered:"
    PRINT "Positive numbers entered: ", countPositive
    PRINT "Negative numbers entered: ", countNegative
    PRINT "Zeroes entered: ", countZero

END
```

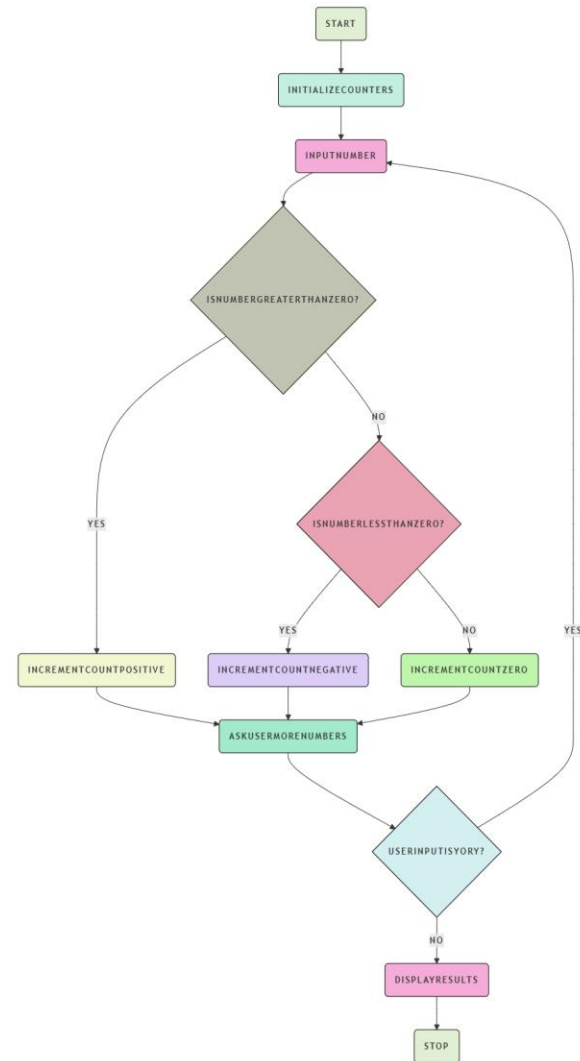
INPUT

```
3.2.1.c > main()
1  #include <stdio.h>
2  int main() {
3      int num, countPositive = 0, countNegative = 0, countZero = 0;
4      char choice;
5      do {
6          printf("Enter a number: ");
7          scanf("%d", &num);
8          if (num > 0)
9              countPositive++;
10         else if (num < 0)
11             countNegative++;
12         else
13             countZero++;
14         printf("Do you want to enter more numbers? (Y/N): ");
15         scanf(" %c", &choice); // space before %c to consume leftover newline
16     } while (choice == 'Y' || choice == 'y');
17     printf(" Positive numbers entered: %d", countPositive);
18     printf(" Negative numbers entered: %d", countNegative);
19     printf(" Zeroes entered: %d", countZero);
20     return 0;
21 }
```

OUTPUT

```
NitishK@Nitishs-MacBook-Air pracricecode % clear
ishkumar/Documents/Code/pracricecode/"3.2.1
Enter a number: 22
Do you want to enter more numbers? (Y/N): 25
Positive numbers entered: 1 Negative numbers entered: 0 Zeroes entered: 0%
● NitishK@Nitishs-MacBook-Air pracricecode % cd "/Users/nitishkumar/Documents/Code/pracricecode/" && gcc 3.2.1.c -o 3.2.1 && "/Users/nit
ishkumar/Documents/Code/pracricecode/"3.2.1
Enter a number: 22
Do you want to enter more numbers? (Y/N): 55
Positive numbers entered: 1 Negative numbers entered: 0 Zeroes entered: 0%
○ NitishK@Nitishs-MacBook-Air pracricecode %
```

FLOWCHART



Experiment-3.2

Activity-2:

C program to print multiplication table of a given number

- **ALGORITHM**
- **Start**
- Declare variables `num` (to store the number) and `i` (loop counter).
- Print a message: *"Enter a number:"*
- Read the number from the user and store it in `num`.
- Print a header: *"Multiplication Table of num"*.
- Print a line separator (for formatting).
- Repeat the following steps using a loop from `i = 1` to `10`:
 Calculate `num * i`.
 Print the result in the format: `num * i = result`.
- End loop.
- **Stop**

PSUEDOCODE:

```
START

DECLARE num, i

PRINT "Enter a number: "
READ num

PRINT "Multiplication Table of num"
PRINT "-----"

FOR i = 1 TO 10 DO
    product = num * i
    PRINT num, "*", i, "=", product
END FOR

STOP
```

INPUT

```
C 3.2.2.c > ...
1  #include <stdio.h>
2  int main() {
3      int num, i;
4      printf("Enter a number: ");
5      scanf("%d", &num);
6      printf("\nMultiplication Table of %d\n", num);
7      printf("-----\n");
8      for (i = 1; i <= 10; i++) {
9          printf("%d * %d = %d\n", num, i, num * i);
10     }
11     return 0;
12 }
```


OUTPUT

```
NitishK@Nitishs-MacBook-Air pracricecode % clear  
ishkumar/Documents/Code/pracricecode/"3.2.2  
Enter a number: 22
```

```
Multiplication Table of 22
```

```
-----
```

```
22 * 1 = 22  
22 * 2 = 44  
22 * 3 = 66  
22 * 4 = 88  
22 * 5 = 110  
22 * 6 = 132  
22 * 7 = 154  
22 * 8 = 176  
22 * 9 = 198  
22 * 10 = 220
```

```
○ NitishK@Nitishs-MacBook-Air pracricecode % █
```

FLOWCHART:



Experiment-3.2

Activity-3(a)

C program to print multiplication table of a given number

- ALGORITHM:
- **Start**
- Declare variables:
- `num` → to store the number entered by the user.
- `i` → loop counter.
- Display message: *"Enter a number:"*.
- Read the value of `num` from the user.
- Print a header: *"Multiplication Table of num"*.
- Print a line separator (for formatting).
- Initialize loop counter `i = 1`.
- **Repeat until `i <= 10`:**
 - a. **Calculate** `result = num × i`.
 - b. Print in format → `num * i = result`.
 - c. Increment `i` by 1.
- End loop.
- **Stop**

PSUEDOCODE:

```
BEGIN
  SET rows = 4

  FOR i = 0 TO rows - 1 DO
    SET val = 1

    FOR j = 0 TO i DO
      PRINT val
      val = val * (i - j) / (j + 1)
    END FOR

    PRINT newline
  END FOR
END
```

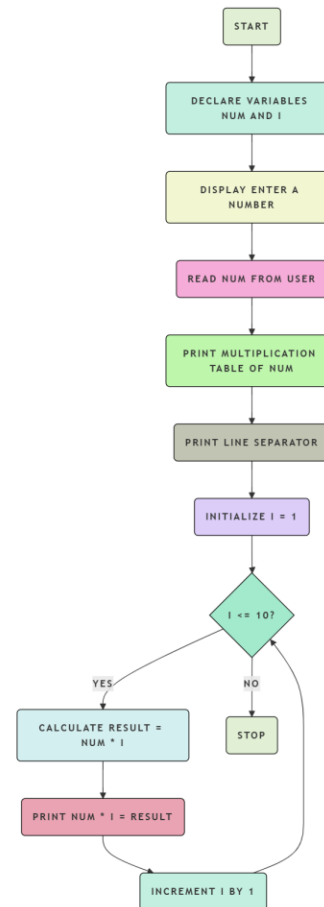
INPUT

```
C 3.2.3.a.c > main()
1  #include <stdio.h>
2  int main() {
3      int rows = 4;
4      for (int i = 0; i < rows; i++) {
5          int val = 1;
6          for (int j = 0; j <= i; j++) {
7              printf("%d ", val);
8              val = val * (i - j) / (j + 1); // Calculate next value
9          }
10         printf("\n");
11     }
12     return 0;
13 }
```

OUTPUT

```
● NitishK@Nitishs-MacBook-Air pracricecode % cd "/Users/nitishkumar/Documents/Code/pracricecode/" && gcc 4_practice_.c -o 4_practice_ && "/Users/nitishkumar/Documents/Code/pracricecode/"4_practice_  
Enter year: 2000  
1st January 2000 was Saturday  
○ NitishK@Nitishs-MacBook-Air pracricecode %
```

FLOWCHART



Experiment-3.2

Activity-3(b)

C program to print Pascal's Triangle

- ALGORITHM
- **Start**
- Initialize variable rows = 4.
- Set loop variable i = 0.
- **Repeat until i < rows:**
 - a. **Set** val = 1.
 - b. Set loop variable j = 0.
 - c. **Repeat until j <= i:**
- Print val.
- Update value: $val = val \times (i - j) / (j + 1)$.
- Increment j by 1.
 - d. Print newline (move to next row).
 - e. Increment i by 1.
- End loop.

PSUEDOCODE:

```
BEGIN
  SET rows = 5

  FOR i = 0 TO rows - 1 DO
    SET val = 1

    FOR j = 0 TO i DO
      PRINT val
      val = val * (i - j) / (j + 1)
    END FOR

    PRINT newline
  END FOR
END
```

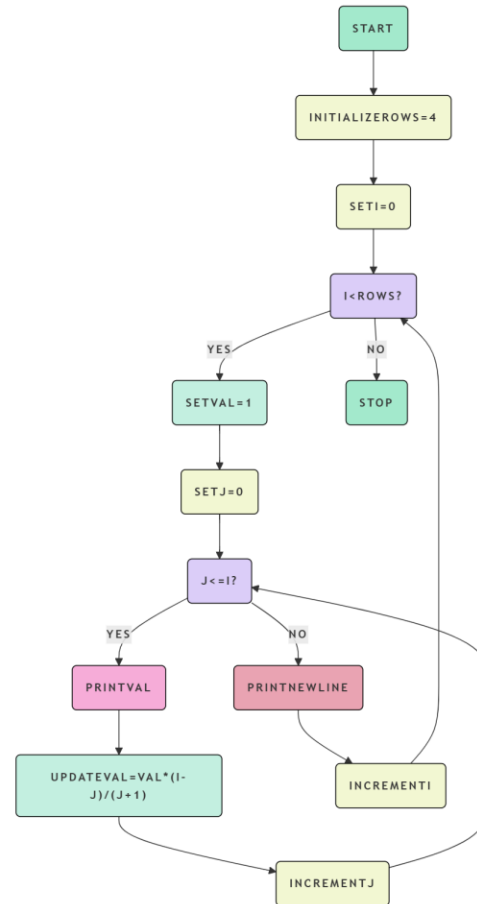
INPUT

```
C 3.2.3.b.c > main()
2  int main() {
4      for (int i = 0; i < rows; i++) {
6          for (int j = 0; j <= i; j++) {
7              printf("%d ", val);
8              val = val * (i - j) / (j + 1);
9          }
10         printf("\n");
11     }
12     return 0;
13 }
```

OUTPUT

```
● NitishK@Nitishs-MacBook-Air Code % cd "/Users/nitishkumar/Documents/Code/" && gcc 3.2.3.b.c -  
o 3.2.3.b && "/Users/nitishkumar/Documents/Code/"3.2.3.b  
1  
1 1  
1 2 1  
1 3 3 1  
1 4 6 4 1  
○ NitishK@Nitishs-MacBook-Air Code %
```

FLOWCHART



Experiment-3.2

Activity-4

Write a program that calculates the population of a town after 10 years, given an initial population of 100,000 and an annual growth rate of 10%. The program should display the population for each year.

- ALGORITHM
- **Start**
- Initialize variables:
`population = 100000` (initial population)
`rate = 0.10` (growth rate = 10%)
`years = 10` (time period in years)
- Print table heading → "Year Population".
- Set loop variable `i = 1`.
- **Repeat until `i <= years`:**
 - Update population** → `population = population × (1 + rate)`.
 - Print → Year (`i`) and Population.
 - Increment `i = i + 1`.
- End loop.
- **Stop**

PSEUDOCODE:

```
START

SET population ← 188888
SET rate ← 8.18
SET years ← 18

PRINT "Year    Population"

FOR i ← 1 TO years DO
    population ← population * (1 + rate)
    PRINT i, population
END FOR

END
```

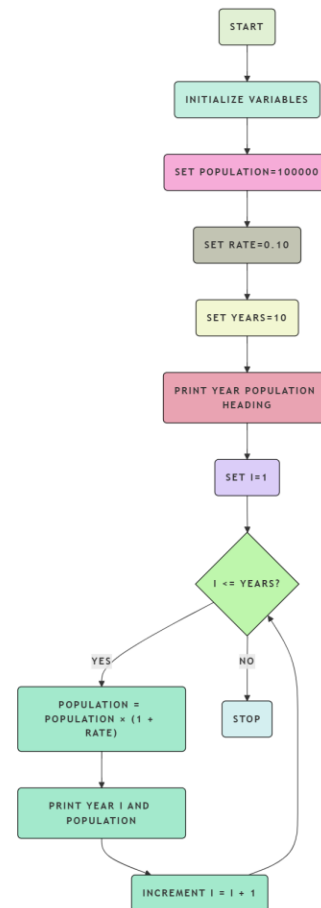
INPUT

```
3.2.4.c > ...
1  #include <stdio.h>
2  #include <math.h>
3  int main() {
4      double population = 100000;
5      double rate = 0.10;
6      int years = 10;
7      printf("Year\tPopulation");
8      for (int i = 1; i <= years; i++) {
9          population = population * (1 + rate);
10         printf("%d\t%.0f", i, population);
11     }
12     return 0;
13 }
```

OUTPUT

```
ents/Code/"3.2.4
Year      Population1      1100002 1210003 1331004 1464105 1610516 1771567 1948728 2143599 235795
10        259374%
○ NitishK@Nitishs-MacBook-Air Code %
```


FLOWCHART



Experiment-3.2

Activity-5

Program to find all numbers that can be expressed as the sum of two cubes in two different ways

- ALGORITHM
- **Start**
- Declare variable `limit`.
- Print $\rightarrow$ "Enter the limit:".
- Read value of `limit` from the user.
- **For** each integer `a` from 1 to $\text{limit}^{(1/3)} \rightarrow$
 - a. **For** each integer `b` from `a` to $\text{limit}^{(1/3)} \rightarrow$
 - i. **For** each integer `c` from `a+1` to $\text{limit}^{(1/3)} \rightarrow$
 - **For** each integer `d` from `c` to $\text{limit}^{(1/3)} \rightarrow$
 - * Compute $\text{sum1} = a^3 + b^3$.
 - * Compute $\text{sum2} = c^3 + d^3$.
 - * If $(\text{sum1} == \text{sum2})$ AND $(\text{sum1} \leq \text{limit}) \rightarrow$
 - Print: $\text{sum1} = a^3 + b^3 = c^3 + d^3$.
- End all loops.
- **Stop**

PSUEDOCODE:

```
BEGIN
  DECLARE limit, a, b, c, d, sum1, sum2

  PRINT "Enter the limit: "
  READ limit

  FOR a = 1 TO a^3 <= limit
    FOR b = a TO b^3 <= limit
      sum1 = a^3 + b^3

      FOR c = a+1 TO c^3 <= limit
        FOR d = c TO d^3 <= limit
          sum2 = c^3 + d^3

          IF sum1 = sum2 AND sum1 <= limit THEN
            PRINT sum1, a, b, c, d
          ENDIF
        ENDFOR
      ENDFOR
    ENDFOR
  ENDFOR

END
```

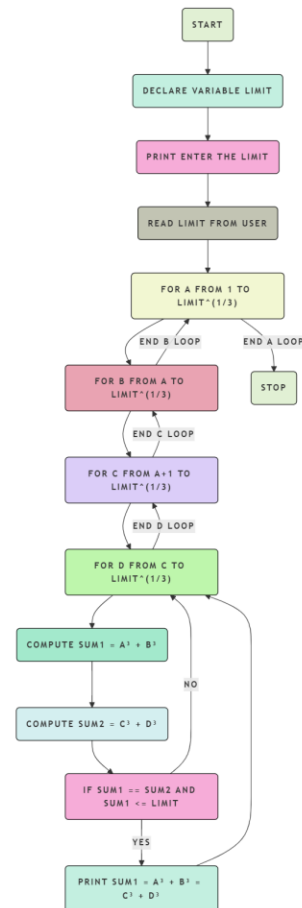
INPUT

```
1  #include <stdio.h>
2  int main() {
3      int limit;
4      printf("Enter the limit: ");
5      scanf("%d", &limit);
6      for (int a = 1; a * a * a <= limit; a++) {
7          for (int b = a; b * b * b <= limit; b++) {
8              for (int c = a + 1; c * c * c <= limit; c++) {
9                  for (int d = c; d * d * d <= limit; d++) {
10                     int sum1 = a*a*a + b*b*b;
11                     int sum2 = c*c*c + d*d*d;
12                     if (sum1 == sum2 && sum1 <= limit) {
13                         printf("%d = %d^3 + %d^3 = %d^3 + %d^3",
14                             sum1, a, b, c, d);
15                     }
16                 }
17             }
18         }
19     }
20     return 0;
21 }
22
```

OUTPUT

```
NitishK@Nitishs-MacBook-Air Code % clear
ents/Code/"3.2.5
Enter the limit: 44
● NitishK@Nitishs-MacBook-Air Code % cd "/Users/nitishkumar/Documents/Code/" && gcc 3.2.5.c -o 3.2.5 && "/Users/nitishkumar/Docum
ents/Code/"3.2.5
Enter the limit: 33cd "/Users/nitishkumar/Documents/Code/" && gcc 3.2.5.c -o 3.2.5 && "/Users/nitishkumar/Documents/Code/"3.2.5
⊗ NitishK@Nitishs-MacBook-Air Code % 22cd "/Users/nitishkumar/Documents/Code/" && gcc 3.2.5.c -o 3.2.5 && "/Users/nitishkumar/Doc
uments/Code/"3.2.5
zsh: command not found: 22cd
○ NitishK@Nitishs-MacBook-Air Code % █
```

FLOWCHART



Experiment 4: Variable and Scope of Variable

1. Declare a global variable outside all functions and use it inside various functions to understand its accessibility.

Algorithm

1. Start
2. Declare a global variable `globalVar` and set it to 10
3. Define function `display()`
Print the value of `globalVar`
4. Define function `modify()`
Change `globalVar` to 20
Print the updated value
5. In `main()`
Print initial value of `globalVar`
Call `display()`
Call `modify()`
Call `display()` again
6. End

PSUEDOCODE:

```
GLOBAL globalVar = 10

FUNCTION display()
    PRINT "Inside display(): globalVar =", globalVar
END FUNCTION

FUNCTION modify()
    globalVar = 20
    PRINT "Inside modify(): globalVar changed to =", globalVar
END FUNCTION

MAIN
    PRINT "Inside main(): initial globalVar =", globalVar
    CALL display()
    CALL modify()
    CALL display()
END MAIN
```

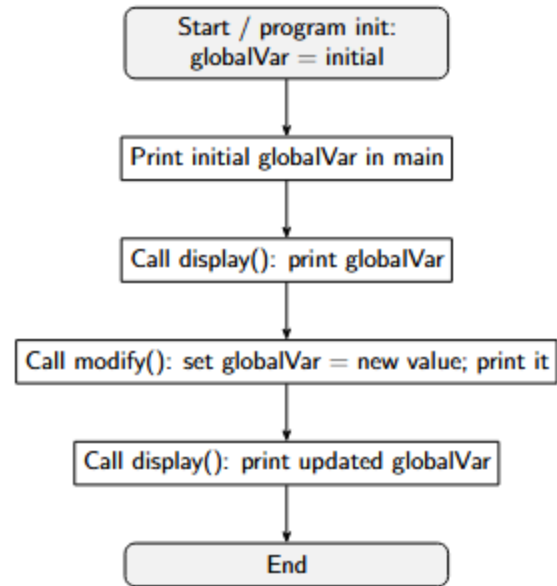

INPUT:

```
1  #include <stdio.h>
2  int globalVar = 10;
3  void display() {
4      printf("Inside display(): globalVar = %d\n", globalVar);
5  }
6  void modify() {
7      globalVar = 20;
8      printf("Inside modify(): globalVar changed to = %d\n", globalVar);
9  }
10 int main() {
11     printf("Inside main(): initial globalVar = %d\n", globalVar);
12     display();
13     modify();
14     display();
15     return 0;
16 }
```

OUTPUT:

```
Inside main(): initial globalVar = 10  
Inside display(): globalVar = 10  
Inside modify(): globalVar changed to = 20  
Inside display(): globalVar = 20  
  
=== Code Execution Successful ===
```

FLOWCHART:



2. Declare a local variable inside a function and try to access it outside the function. Compare this with accessing the global variable from within the function.

Algorithm

1. Start
2. Declare a global variable `globalVar = 100`
3. Define function `testLocal()`
 - a. Declare a local variable `localVar = 50`
 - b. Print value of `localVar`
 - c. Print value of `globalVar`
4. Define function `testAccess()`
 - a. Print value of `globalVar`
5. In `main()`
 - a. Call `testLocal()`
 - b. Call `testAccess()`
6. End

PSUEDOCODE:

```
GLOBAL globalVar = 100

FUNCTION testLocal
    localVar = 50
    PRINT "Inside testLocal(): localVar =", localVar
    PRINT "Inside testLocal(): globalVar =", globalVar
END FUNCTION

FUNCTION testAccess
    PRINT "Inside testAccess(): globalVar =", globalVar
END FUNCTION

MAIN
    CALL testLocal
    CALL testAccess
END MAIN
```

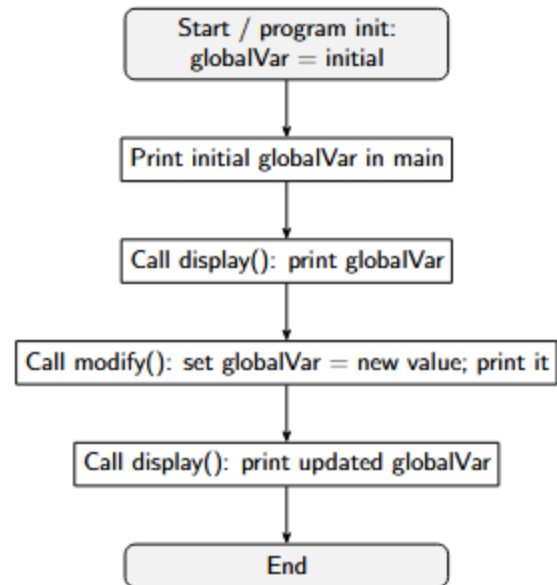
INPUT:

```
1  #include <stdio.h>
2  int globalVar = 100;
3+ void testLocal() {
4      int localVar = 50;
5      printf("Inside testLocal(): localVar = %d\n", localVar);
6      printf("Inside testLocal(): globalVar = %d\n", globalVar);
7  }
8+ void testAccess() {
9      printf("Inside testAccess(): globalVar = %d\n", globalVar);
10 }
11- int main() {
12     testLocal();
13     testAccess();
14     return 0;
15 }
16
```

OUTPUT:

```
Inside testLocal(): localVar = 50  
Inside testLocal(): globalVar = 100  
Inside testAccess(): globalVar = 100  
  
=== Code Execution Successful ===
```

FLOWCHART:



3. Declare variables within different code blocks (enclosed by curly braces) and test their accessibility within and outside those blocks.

Algorithm

1. Start
2. Declare an integer variable a and assign value 10
3. Print the value of a
4. Begin **Block 1**
 1. Declare integer variable b and assign value 20
 2. Print a
 3. Print b
5. End **Block 1**
6. Begin **Block 2**
 1. Declare integer variable c and assign value 30
 2. Print a
 3. Print c
7. End **Block 2**
8. End program

PSUEDOCODE:

```
BEGIN
    a ← 10
    PRINT "In main block: a =", a

    BEGIN BLOCK 1
        b ← 20
        PRINT "Inside Block 1: a =", a
        PRINT "Inside Block 1: b =", b
    END BLOCK 1

    BEGIN BLOCK 2
        c ← 30
        PRINT "Inside Block 2: a =", a
        PRINT "Inside Block 2: c =", c
    END BLOCK 2

END
```

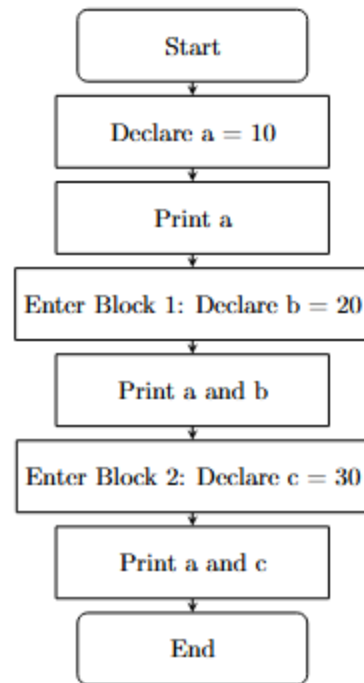
INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int a = 10; // variable in main block
4      printf("In main block: a = %d\n", a);
5      {
6          int b = 20;
7          printf("Inside Block 1: a = %d\n", a);
8          printf("Inside Block 1: b = %d\n", b);
9      }
10     {
11         int c = 30;
12         printf("Inside Block 2: a = %d\n", a);
13         printf("Inside Block 2: c = %d\n", c);
14     }
15     return 0;
16 }
```

OUTPUT:

```
In main block: a = 10  
Inside Block 1: a = 10  
Inside Block 1: b = 20  
Inside Block 2: a = 10  
Inside Block 2: c = 30  
  
=== Code Execution Successful ===
```

FLOWCHART:



4. Declare a static local variable inside a function. Observe how its value persists across function calls.

Algorithm

1. Start the program.
2. Define a function named counter.
3. Inside the function, declare a static variable count initialized to 0.
4. Increment count by 1.
5. Display the value of count.
6. In the main program, call the function counter three times.
7. Stop the program.

PSEUDOCODE:

```
FUNCTION counter
    DECLARE static count = 0
    INCREMENT count
    PRINT count
END FUNCTION

START
    CALL counter
    CALL counter
    CALL counter
STOP
```

INPUT:

```
1  #include <stdio.h>
2  void counter() {
3      static int count = 0;
4      count++;
5      printf("Counter value: %d\n", count);
6  }
7  int main() {
8      counter();
9      counter();
10     counter();
11     return 0;
12 }
```


OUTPUT:

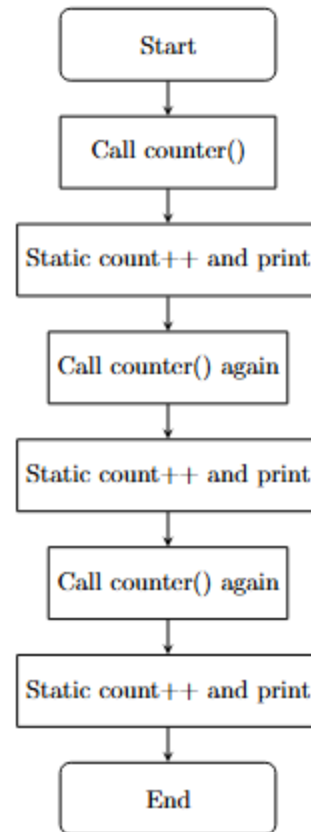
```
Counter value: 1
```

```
Counter value: 2
```

```
Counter value: 3
```

```
=== Code Execution Successful ===
```

FLOWCHART:



Experiment 5: Array

1. WAP to read a list of integers and store it in a single dimensional array. Write a C program to print the second largest integer in a list of integers.

ALGORITHM

1. To find the second largest number in an array
2. Start
3. Declare n
4. Read the value of n
5. Declare array arr of size n
6. Read all n integers into the array
7. Set `largest = arr[0]`
8. Set `second_largest = arr[0]`
9. For each element from index 1 to n-1:
 1. If `arr[i] > largest`
 1. `second_largest = largest`
 2. `largest = arr[i]`
 2. Else if `arr[i] > second_largest` and `arr[i] != largest`
 1. `second_largest = arr[i]`
10. If `second_largest == largest`
Print "Second largest does not exist"
Else
Print second largest value
11. Stop

PSEUDOCODE:

```
BEGIN
  INPUT n
  DECLARE array arr of size n

  FOR i FROM 0 TO n-1
    INPUT arr[i]
  END FOR

  SET largest = arr[0]
  SET second_largest = arr[0]

  FOR i FROM 1 TO n-1
    IF arr[i] > largest THEN
      second_largest = largest
      largest = arr[i]
    ELSE IF arr[i] > second_largest AND arr[i] != largest THEN
      second_largest = arr[i]
    END IF
  END FOR

  IF second_largest == largest THEN
    PRINT "Second largest does not exist"
  ELSE
    PRINT "Second largest integer is: ", second_largest
  END IF
END
```

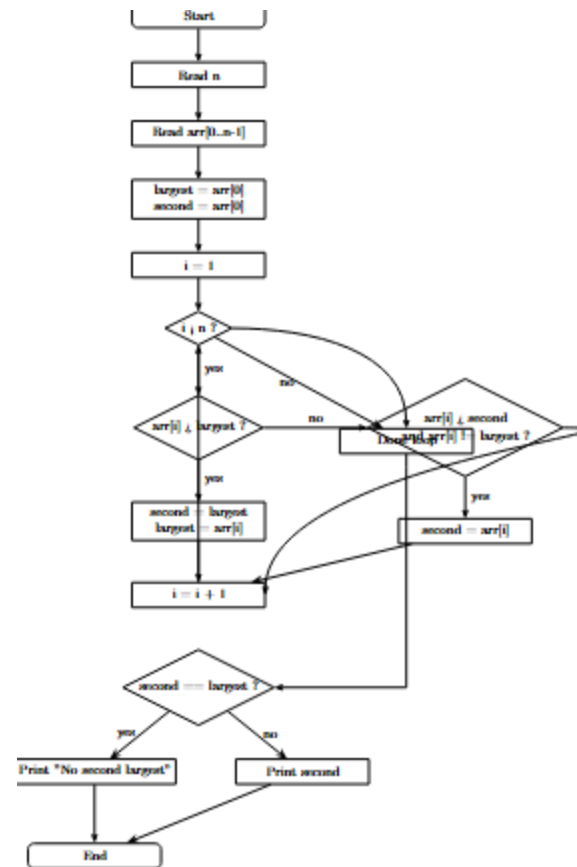
INPUT:

```
1 #include <stdio.h>
2 int main() {
3     int n;
4
5     printf("Enter the number of elements: ");
6     scanf("%d", &n);
7
8     int arr[n];
9
10    printf("Enter %d integers:\n", n);
11    for (int i = 0; i < n; i++) {
12        scanf("%d", &arr[i]);
13    }
14
15    int largest = arr[0];
16    int second_largest = arr[0];
17
18
19    for (int i = 1; i < n; i++) {
20        if (arr[i] > largest) {
21            second_largest = largest;
22            largest = arr[i];
23        } else if (arr[i] > second_largest && arr[i] != largest) {
24            second_largest = arr[i];
25        }
26    }
27
28
29    if (second_largest == largest) {
30        printf("Second largest does not exist (all numbers equal)\n");
31    } else {
32        printf("Second largest integer is: %d\n", second_largest);
33    }
34
35    return 0;
36 }
```

OUTPUT:

```
Enter the number of elements: 4  
Enter 4 integers:  
4 5 6 7  
Second largest integer is: 6  
  
=== Code Execution Successful ===
```

FLOWCHART:



2.WAP to read a list of integers and store it in a single dimensional array. Write a C program to count and display positive, negative, odd, and even numbers in an array.

ALGORITHM:

1. Start
2. Declare an integer n
3. Ask the user to enter the number of elements
4. Read n
5. Declare an integer array of size n
6. Read all n integers into the array
7. Initialize counters:
8. `positive = 0`
9. `negative = 0`
10. `odd = 0`
11. `even = 0`
12. Loop through each element of the array
13. If `element ≥ 0` → increment `positive`
14. Else → increment `negative`
15. If `element % 2 == 0` → increment `even`
16. Else → increment `odd`
17. Display the values of all four counters
18. Stop

12/7/2025

PSUEDOCODE:

```
BEGIN
  DECLARE n, i
  PRINT "Enter the number of elements"
  READ n

  DECLARE array of size n

  PRINT "Enter n integers"
  FOR i = 0 TO n-1
    READ arr[i]
  END FOR

  SET positive = 0
  SET negative = 0
  SET odd = 0
  SET even = 0

  FOR i = 0 TO n-1
    IF arr[i] >= 0 THEN
      positive = positive + 1
    ELSE
      negative = negative + 1
    ENDIF
    IF arr[i] MOD 2 == 0 THEN
      even = even + 1
    ELSE
      odd = odd + 1
    ENDIF
  END FOR

  PRINT "Positive numbers:", positive
  PRINT "Negative numbers:", negative
  PRINT "Odd numbers:", odd
  PRINT "Even numbers:", even
END
```

INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int n;
4      printf("Enter the number of elements: ");
5      scanf("%d", &n);
6
7      int arr[n];
8
9      printf("Enter %d integers:\n", n);
10     for (int i = 0; i < n; i++) {
11         scanf("%d", &arr[i]);
12     }
13     int positive = 0, negative = 0, odd = 0, even = 0;
14     for (int i = 0; i < n; i++) {
15         if (arr[i] >= 0)
16             positive++;
17         else
18             negative++;
19
20         if (arr[i] % 2 == 0)
21             even++;
22         else
23             odd++;
24     }
25     printf("\n--- Result ---\n");
26     printf("Positive numbers: %d\n", positive);
27     printf("Negative numbers: %d\n", negative);
28     printf("Odd numbers: %d\n", odd);
29     printf("Even numbers: %d\n", even);
30     return 0;
```

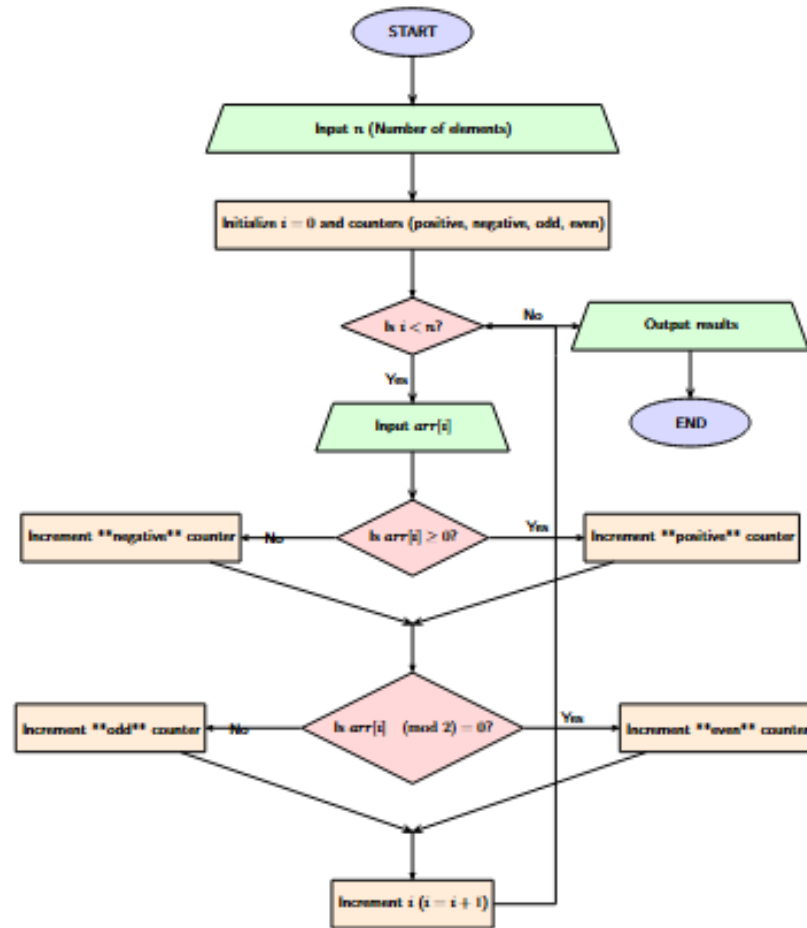
OUTPUT:

```
Enter the number of elements: 4
Enter 4 integers:
3 4 5 6

--- Result ---
Positive numbers: 4
Negative numbers: 0
Odd numbers: 2
Even numbers: 2

=== Code Execution Successful ===
```

FLOWCHART:



3. WAP to read a list of integers and store it in a single dimensional array. Write a C program to find the frequency of a particular number in a list of integers.

- **ALGORITHM**

- **Step 1:** Start

Step 2: Declare integers n, key, count and an array arr[]

Step 3: Set count = 0

Step 4: Read the value of n (number of elements)

Step 5: Read n integers into the array arr[]

Step 6: Read the number key whose frequency is to be found

Step 7: Repeat for i = 0 to n - 1:

- If arr[i] == key, increment count

Step 8: Display the frequency

Step 9: End

PSEUDOCODE:

```
BEGIN

    SET count ← 0

    OUTPUT "Enter the number of elements"
    INPUT n

    DECLARE array arr of size n

    OUTPUT "Enter n integers"
    FOR i ← 0 TO n-1 DO
        INPUT arr[i]
    END FOR

    OUTPUT "Enter the number whose frequency you want to find"
    INPUT key

    FOR i ← 0 TO n-1 DO
        IF arr[i] = key THEN
            count ← count + 1
        END IF
    END FOR

    OUTPUT "Frequency of", key, "is", count

END
```

INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int n, key, count = 0;
4      printf("Enter the number of elements: ");
5      scanf("%d", &n);
6      int arr[n];
7      printf("Enter %d integers:\n", n);
8      for (int i = 0; i < n; i++) {
9          scanf("%d", &arr[i]);
10     }
11     printf("Enter the number whose frequency you want to find: ");
12     scanf("%d", &key);
13     for (int i = 0; i < n; i++) {
14         if (arr[i] == key)
15             count++;
16     }
17     printf("\nFrequency of %d is: %d\n", key, count);
18
19     return 0;
20 }
```

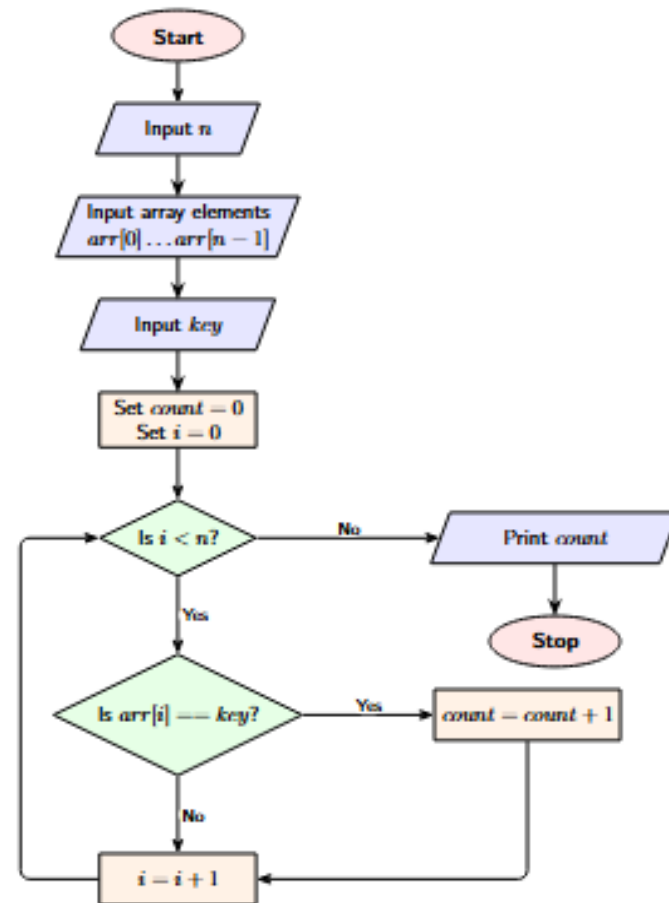
OUTPUT:

```
Enter the number of elements: 4
Enter 4 integers:
1 2 3 4
Enter the number whose frequency you want to find: 3

Frequency of 3 is: 1

=== Code Execution Successful ===
```


FLOWCHART:



4.WAP that reads two matrices A ($m \times n$) and B ($p \times q$) and computes the product A and B. Read matrix A and matrix B in row major order respectively. Print both the input matrices and resultant matrix with suitable headings and output should be in matrix format only. Program must check the compatibility of orders of the matrices for multiplication. Report appropriate message in case of incompatibility.

ALGORITHM:

1. Read m and n .
2. Read p and q .
3. If $n \neq p$ then print incompatibility message and stop.
4. Allocate matrices A ($m \times n$), B ($p \times q$) and C ($m \times q$).
5. Read elements of A in row-major order.
6. Read elements of B in row-major order.
7. For each i from 0 to $m-1$:
8. For each j from 0 to $q-1$:
Set $C[i][j] = 0$.
For each k from 0 to $n-1$:
 - $C[i][j] += A[i][k] * B[k][j]$.
 - Print Matrix A with heading.
 - Print Matrix B with heading.
 - Print Resultant Matrix C with heading

PSUEDOCODE:

```
BEGIN
  READ m, n
  READ p, q

  IF n != p THEN
    PRINT "Matrix multiplication NOT possible!"
    PRINT "Reason: Columns of A (n) != Rows of B (p)"
    STOP
  ENDIF

  DECLARE A[m][n], B[p][q], C[m][q]

  PRINT "Enter elements of Matrix A (m x n) in row-major order:"
  FOR i = 0 TO m-1 DO
    FOR j = 0 TO n-1 DO
      READ A[i][j]
    ENDFOR
  ENDFOR

  PRINT "Enter elements of Matrix B (p x q) in row-major order:"
  FOR i = 0 TO p-1 DO
    FOR j = 0 TO q-1 DO
      READ B[i][j]
    ENDFOR
  ENDFOR

  FOR i = 0 TO m-1 DO
    FOR j = 0 TO q-1 DO
      C[i][j] = 0
      FOR k = 0 TO n-1 DO
        C[i][j] = C[i][j] + A[i][k] * B[k][j]
      ENDFOR
    ENDFOR
  ENDFOR

  PRINT "Matrix A (m x n):"
  FOR i = 0 TO m-1 DO
    FOR j = 0 TO n-1 DO
      PRINT A[i][j] WITH SPACING
    ENDFOR
    PRINT NEWLINE
  ENDFOR

  PRINT "Matrix B (p x q):"
  FOR i = 0 TO p-1 DO
    FOR j = 0 TO q-1 DO
      PRINT B[i][j] WITH SPACING
    ENDFOR
    PRINT NEWLINE
  ENDFOR
```

INPUT:

```
1 #include <stdio.h>
2 int main() {
3     int m, n, p, q;
4     printf("Enter the order of Matrix A (m n): ");
5     scanf("%d %d", &m, &n);
6     printf("Enter the order of Matrix B (p q): ");
7     scanf("%d %d", &p, &q);
8     if (n != p) {
9         printf("\nMatrix multiplication NOT possible!\n");
10        printf("Reason: Columns of A (%d) != Rows of B (%d)\n", n, p);
11        return 0;
12    }
13    int A[m][n], B[p][q], C[m][q];
14    printf("\nEnter elements of Matrix A (%d x %d) in row-major order:\n", m, n);
15    for (int i = 0; i < m; i++)
16        for (int j = 0; j < n; j++)
17            scanf("%d", &A[i][j]);
18
19    printf("\nEnter elements of Matrix B (%d x %d) in row-major order:\n", p, q);
20    for (int i = 0; i < p; i++)
21        for (int j = 0; j < q; j++)
22            scanf("%d", &B[i][j]);
23
24    for (int i = 0; i < m; i++) {
25        for (int j = 0; j < q; j++) {
26            C[i][j] = 0;
27            for (int k = 0; k < n; k++)
28                C[i][j] += A[i][k] * B[k][j];
29        }
30    }
31
32    printf("\nMatrix A (%d x %d):\n", m, n);
33    for (int i = 0; i < m; i++) {
34        for (int j = 0; j < n; j++)
35            printf("%d ", A[i][j]);
36        printf("\n");
37    }
38
39    printf("\nMatrix B (%d x %d):\n", p, q);
40    for (int i = 0; i < p; i++) {
41        for (int j = 0; j < q; j++)
42            printf("%d ", B[i][j]);
43        printf("\n");
44    }
45 }
```

OUTPUT:

```
Enter the order of Matrix A (m n): 2 2
Enter the order of Matrix B (p q): 2 2

Enter elements of Matrix A (2 * 2) in row-major order:
2 2
2 2

Enter elements of Matrix B (2 * 2) in row-major order:
2
2 2
2

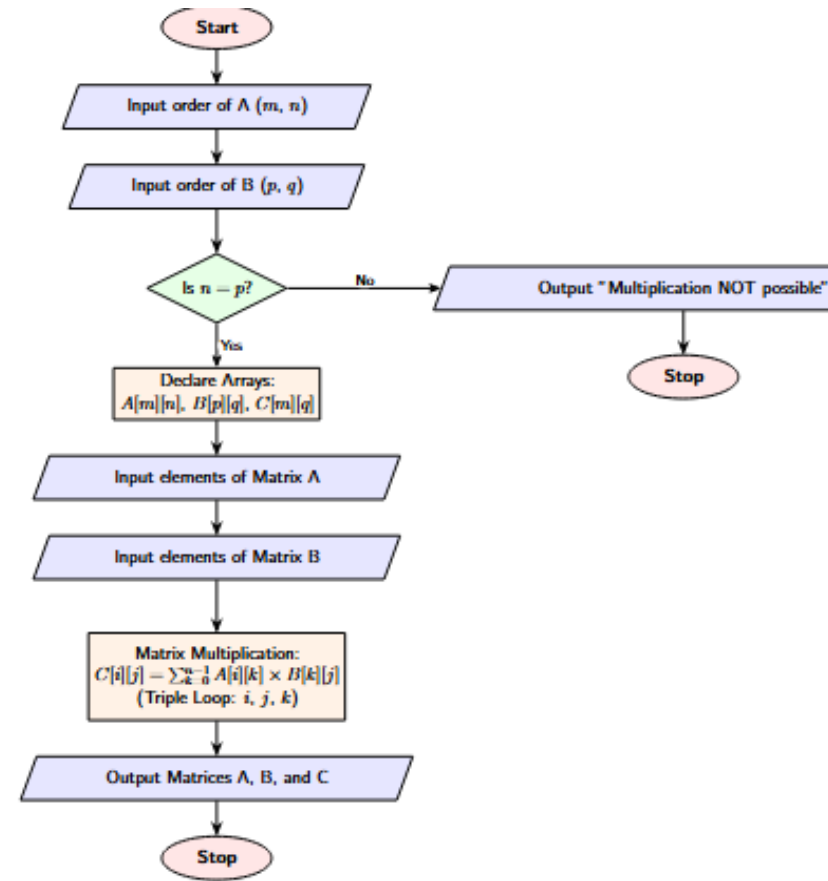
Matrix A (2 * 2):
  2  2
  2  2

Matrix B (2 * 2):
  2  2
  2  2

Resultant Matrix (A * B) (2 * 2):
  8  8
  8  8

=== Code Execution Successful ===
```

FLOWCHART:



Experiment 6

1. Develop a recursive and non-recursive function $FACT(num)$ to find the factorial of a number, $n!$, defined by $FACT(n) = 1$, if $n = 0$. Otherwise, $FACT(n) = n \times FACT(n-1)$. Using this function, write a C program to compute the binomial coefficient. Tabulate the results for different values of n and r with suitable messages.

ALGORITHM:

1. Start
2. Read the values of **n** and **r**
3. If **r > n**, print "Invalid input" and stop
4. Compute **n!** using:
 5. recursive $FACT(n)$
 6. non-recursive $FACT(n)$
7. Compute **r!** and **(n-r)!** Similarly
8. Display **nCr**
9. Create a table by running loops:
 1. For $i = 0$ to n
 1. For $j = 0$ to i
 1. Compute iCj using factorial function
 2. Print the value
10. Stop

PSUEDOCODE:

```
FUNCTION FACT_RECURSIVE(n)
    IF n == 0 THEN
        RETURN 1
    ELSE
        RETURN n * FACT_RECURSIVE(n - 1)
    ENDIF
END FUNCTION
```


INPUT:

```
1  #include <stdio.h>
2  long fact_rec(long n) {
3      if (n == 0)
4          return 1;
5      return n * fact_rec(n - 1);
6  long fact_nonrec(long n) {
7      long f = 1;
8      for (long i = 1; i <= n; i++)
9          f *= i;
10     return f;
11 long binomial(long n, long r) {
12     return fact_rec(n) / (fact_rec(r) * fact_rec(n - r));
13 int main() {
14     int n, r;
15     printf("Enter n value: ");
16     scanf("%d", &n);
17     printf("Enter r value: ");
18     scanf("%d", &r);
19     if (r > n) {
20         printf("Invalid input! r cannot be greater than n.\n");
21         return 0;
22     printf("\n--- FACTORIAL RESULTS ---\n");
23     printf("Recursive FACT(%d) = %ld\n", n, fact_rec(n));
24     printf("Non-recursive FACT(%d) = %ld\n", n, fact_nonrec(n));
25     long ncr = binomial(n, r);
26     printf("\n--- BINOMIAL COEFFICIENT ---\n");
27     printf("C(%d, %d) = %ld\n", n, r, ncr);
28     printf("\n--- TABLE OF nCr FOR n UP TO %d ---\n", n);
29     printf("  n    r    nCr\n");
30     printf("-----\n");
```

OUTPUT:

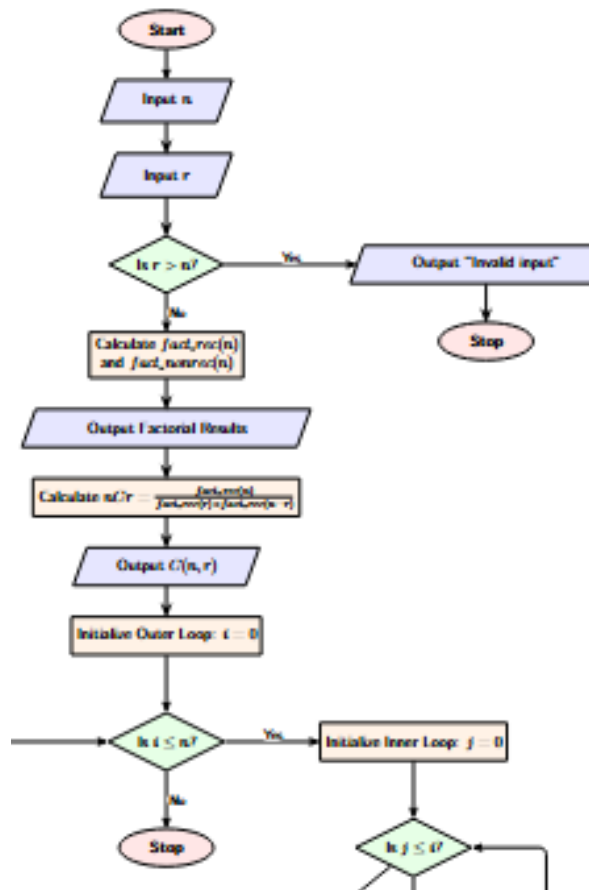
```
Enter n value: 4
Enter r value: 4

--- FACTORIAL RESULTS ---
Recursive FACT(4)      = 24
Non-recursive FACT(4) = 24

--- BINOMIAL COEFFICIENT ---
C(4, 4) = 1

--- TABLE OF nCr FOR n UP TO 4 ---
  n   r   nCr
-----
  0   0     1
  1   0     1
  1   1     1
  2   0     1
  2   1     2
  2   2     1
  3   0     1
  3   1     3
  3   2     3
  3   3     1
  4   0     1
  4   1     4
  4   2     6
  4   3     4
  4   4     1
```

FLOWCHART:



2. Develop a recursive function GCD (num1, num2) that accepts two integer arguments. Write a C program that invokes this function to find the greatest common divisor of two given integers.

- ALGORITHM:
- **Step 1:** Start
 - Step 2:** Read two integers num1 and num2
 - Step 3:** Call the recursive function gcd(num1, num2)
 - Step 4:** Inside the function:
 - If $b == 0$, return a
 - Otherwise return gcd(b, a % b)
 - Step 5:** Display the returned GCD value
 - Step 6:** Stop

PSUEDOCODE:

```
FUNCTION gcd(a, b)
    IF b = 0 THEN
        RETURN a
    ELSE
        RETURN gcd(b, a MOD b)
    ENDIF
END FUNCTION

START
    PRINT "Enter two integers: "
    READ num1, num2

    result ← gcd(num1, num2)

    PRINT "Greatest Common Divisor (GCD) of ", num1, " and ", num2, " = ", result
END
```

INPUT:

```
1  #include <stdio.h>
2
3  int gcd(int a, int b) {
4      if (b == 0)
5          return a;
6      return gcd(b, a % b);
7  }
8
9  int main() {
10     int num1, num2;
11
12     printf("Enter two integers: ");
13     scanf("%d %d", &num1, &num2);
14
15     printf("Greatest Common Divisor (GCD) of %d and %d = %d\n",
16           num1, num2, gcd(num1, num2));
17
18     return 0;
19 }
```

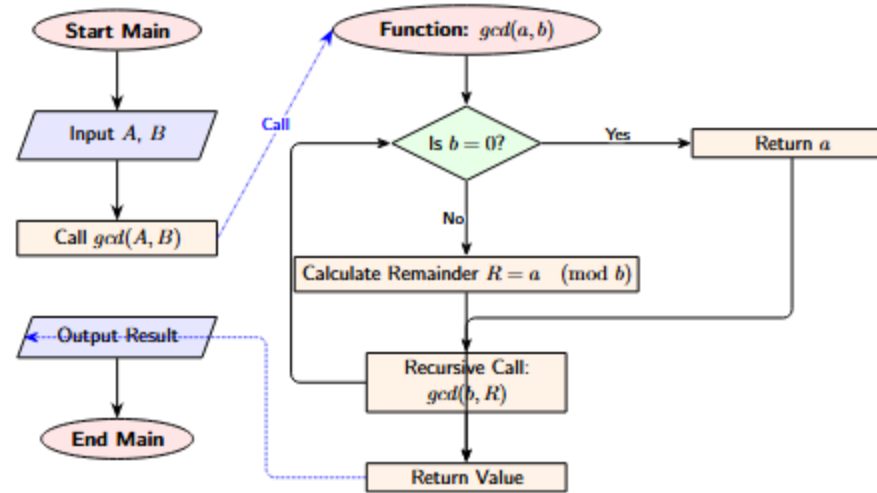
OUTPUT:

```
Enter two integers: 2 3
```

```
Greatest Common Divisor (GCD) of 2 and 3 = 1
```

```
=== Code Execution Successful ===
```

FLOWCHART:



3. Develop a recursive function FIBO(num) that accepts an integer argument. Write a C program that invokes this function to generate the Fibonacci sequence up to num.

- **ALGORITHM**

- **Step 1:** Start

- Step 2:** Define a recursive function FIBO(num)

- If num = 0 → return 0

- If num = 1 → return 1

- Else return FIBO(num - 1) + FIBO(num - 2)

- Step 3:** In main(), declare integer n

- Step 4:** Ask the user to enter the number of terms

- Step 5:** Read n

- Step 6:** Print message "Fibonacci sequence up to n terms"

- Step 7:** Loop i from 0 to n-1

- Call FIBO(i) and print result

- Step 8:** En

PSUEDOCODE:

```
FUNCTION FIBO(num)
    IF num == 0 THEN
        RETURN 0
    ELSE IF num == 1 THEN
        RETURN 1
    ELSE
        RETURN FIBO(num - 1) + FIBO(num - 2)
    END IF
END FUNCTION

BEGIN
    DECLARE n
    PRINT "Enter the number of terms:"
    READ n

    PRINT "Fibonacci sequence up to n terms:"

    FOR i ← 0 TO n-1 DO
        PRINT FIBO(i)
    END FOR
END
```

INPUT:

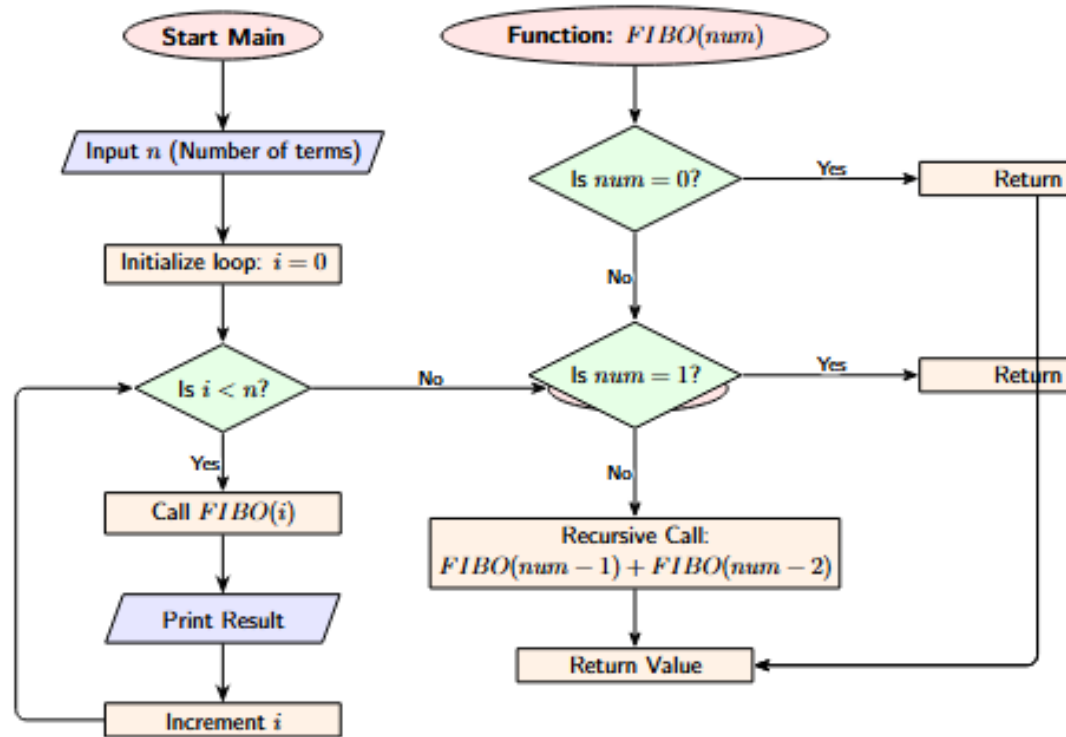
```
1  #include <stdio.h>
2
3  int FIBO(int num) {
4      if (num == 0)
5          return 0;
6      if (num == 1)
7          return 1;
8      return FIBO(num - 1) + FIBO(num - 2);
9  }
10
11 int main() {
12     int n;
13
14     printf("Enter the number of terms: ");
15     scanf("%d", &n);
16
17     printf("Fibonacci sequence up to %d terms:\n", n);
18     for (int i = 0; i < n; i++) {
19         printf("%d ", FIBO(i));
20     }
21
22     return 0;
23 }
```

OUTPUT:

```
Enter the number of terms: 5
Fibonacci sequence up to 5 terms:
0 1 1 2 3

=== Code Execution Successful ===
```

FLOWCHART:



4. Develop a C function ISPRIME(num) that accepts an integer argument and returns 1 if the argument is prime, and 0 otherwise. Write a C program that invokes this function to generate prime numbers between the given ranges

- ALGORITHM:
- **Step 1:**Start.
- **Step 2:**Declare two integer variables: low and high.
- **Step 3:**Read the values of low and high from the user.
- **Step 4:**Display a message indicating that prime numbers within the given range will be printed.
- **Step 5:**Repeat for each number i from low to high:
 - Call the function **ISPRIME(i)**.
 - If ISPRIME(i) returns **1**, print the number i.
- **Step 6:**
- End.

PSUEDOCODE:

```
FUNCTION ISPRIME(num)
    IF num < 2 THEN
        RETURN 0
    END IF
    FOR i FROM 2 TO floor(sqrt(num)) DO
        IF num % i == 0 THEN
            RETURN 0
        END IF
    END FOR
    RETURN 1
END FUNCTION

MAIN
    READ low, high
    PRINT "Prime numbers between low and high are:"
    FOR n FROM low TO high DO
        IF ISPRIME(n) == 1 THEN
            PRINT n
        END IF
    END FOR
END MAIN
```

INPUT:

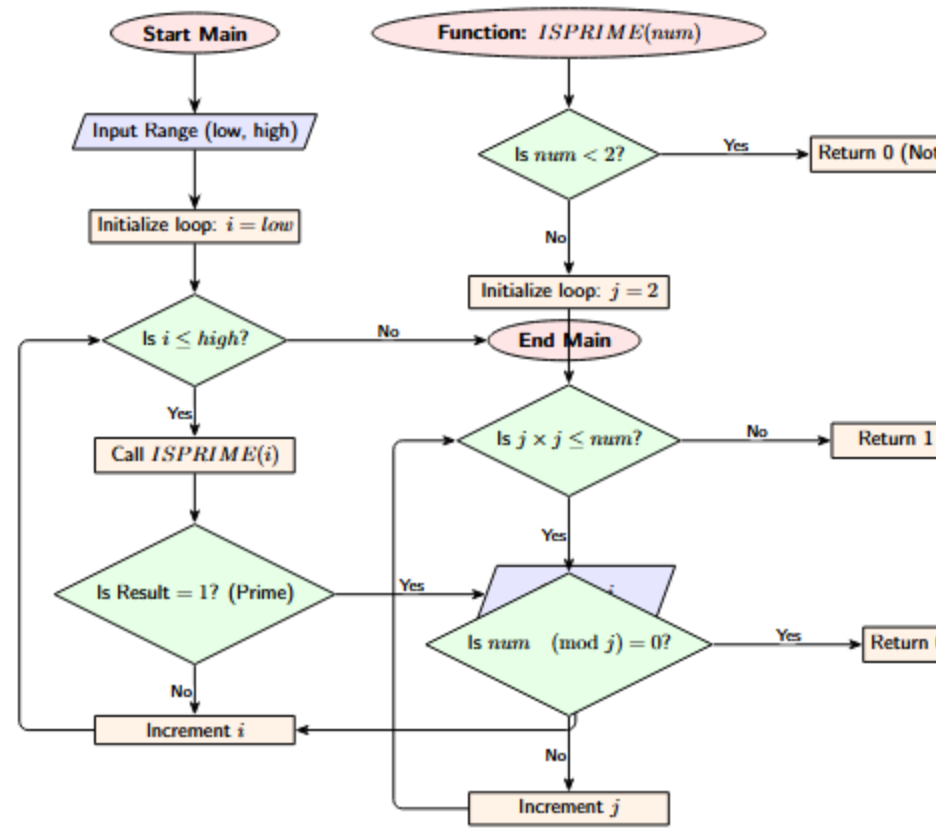
```
1  #include <stdio.h>
2  int ISPRIME(int num) {
3      if (num < 2)
4          return 0;
5      for (int i = 2; i * i <= num; i++) {
6          if (num % i == 0)
7              return 0;
8      }
9      return 1;
10 }
11 int main() {
12     int low, high;
13     printf("Enter the range (low high): ");
14     scanf("%d %d", &low, &high);
15     printf("Prime numbers between %d and %d are:\n", low, high);
16     for (int i = low; i <= high; i++) {
17         if (ISPRIME(i))
18             printf("%d ", i);
19     }
20     return 0;
21 }
```


OUTPUT:

```
Enter the range (low high): 7 8
Prime numbers between 7 and 8 are:
7

=== Code Execution Successful ===
```

FLOWCHART:



5. DEVELOP A FUNCTION REVERSE STR THAT ACCEPTS A STRING ARGUMENT. WRITE A C PROGRAM THAT INVOKES THIS FUNCTION TO FIND THE REVERSE OF A GIVEN STRING.

ALGORITHM:

1. Start
2. Declare a character array str
3. Prompt the user to enter a string
4. Read the input string
5. Initialize two index variables:
 1. $i = 0$
 2. $j = \text{length of string} - 1$
6. Repeat while $i < j$
 - a. Swap characters at positions i and j
 - b. Increment i
 - c. Decrement j
7. After the loop, the string is reversed
8. Display the reversed string
9. End

PSUEDOCODE:

```
BEGIN

    DECLARE string str
    PRINT "Enter a string:"
    READ str

    SET i = 0
    SET j = length(str) - 1

    WHILE i < j DO
        temp = str[i]
        str[i] = str[j]
        str[j] = temp
        i = i + 1
        j = j - 1
    END WHILE

    PRINT "Reversed string: ", str

END
```

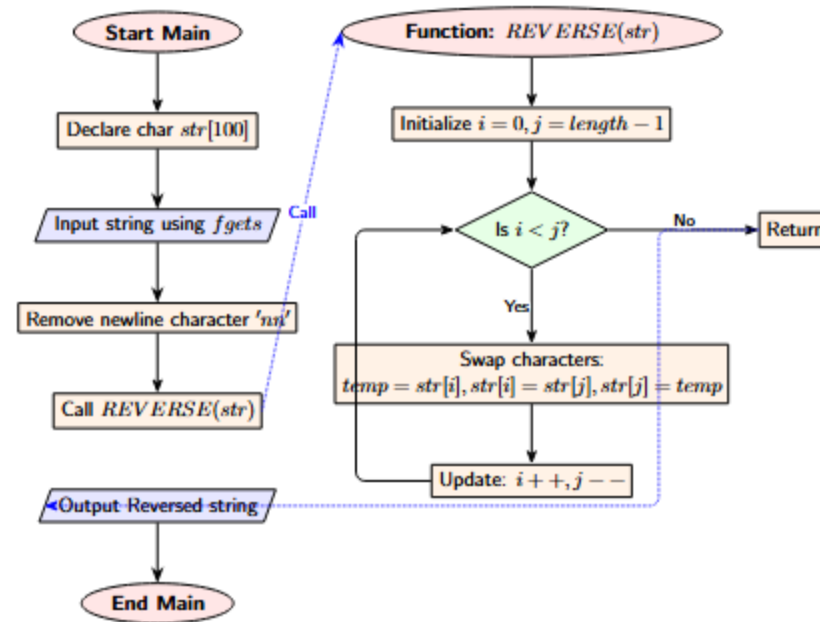
INPUT:

```
1  #include <stdio.h>
2  #include <string.h>
3  void REVERSE(char str[]) {
4      int i, j;
5      char temp;
6      for (i = 0, j = strlen(str) - 1; i < j; i++, j--) {
7          temp = str[i];
8          str[i] = str[j];
9          str[j] = temp;
10     }
11 }
12 int main() {
13     char str[100];
14     printf("Enter a string: ");
15     fgets(str, sizeof(str), stdin);
16     str[strcspn(str, "\n")] = '\0';
17     REVERSE(str);
18     printf("Reversed string: %s\n", str);
19     return 0;
20 }
```

OUTPUT:

```
Enter a string: SIDDHANT  
Reversed string: TNAHDDIS  
  
=== Code Execution Successful ===
```

FLOWCHART:



Experiment 7: Structures and Union

- Write a C program that uses functions to perform the following operations:
 - a. Reading a complex number.

ALGORITHM:

1. Start
2. Declare a structure `Complex` with two float members: `real` and `imag`.
3. Define a function `readComplex()`
 1. Inside the function:
 1. Declare a variable `c` of type `Complex`.
 2. Ask the user to enter the real part.
 3. Read and store it in `c.real`.
 4. Ask the user to enter the imaginary part.
 5. Read and store it in `c.imag`.
 6. Return structure `c`.
4. In `main()` function:
 1. Declare a variable `c` of type `Complex`.
 2. Display a message "Reading a complex number".
 3. Call `readComplex()` and store the returned structure in `c`.
 4. Print the complex number in the form **a + bi**.
5. End.

PSUEDOCODE:

```
START

DECLARE structure Complex
    real : float
    imag : float
END structure

FUNCTION readComplex()
    DECLARE c as Complex
    PRINT "Enter real part: "
    INPUT c.real
    PRINT "Enter imaginary part: "
    INPUT c.imag
    RETURN c
END FUNCTION

DECLARE c as Complex

PRINT "Reading a complex number:"
c ← readComplex()

PRINT "You entered: ", c.real, " + ", c.imag, "i"

END
```

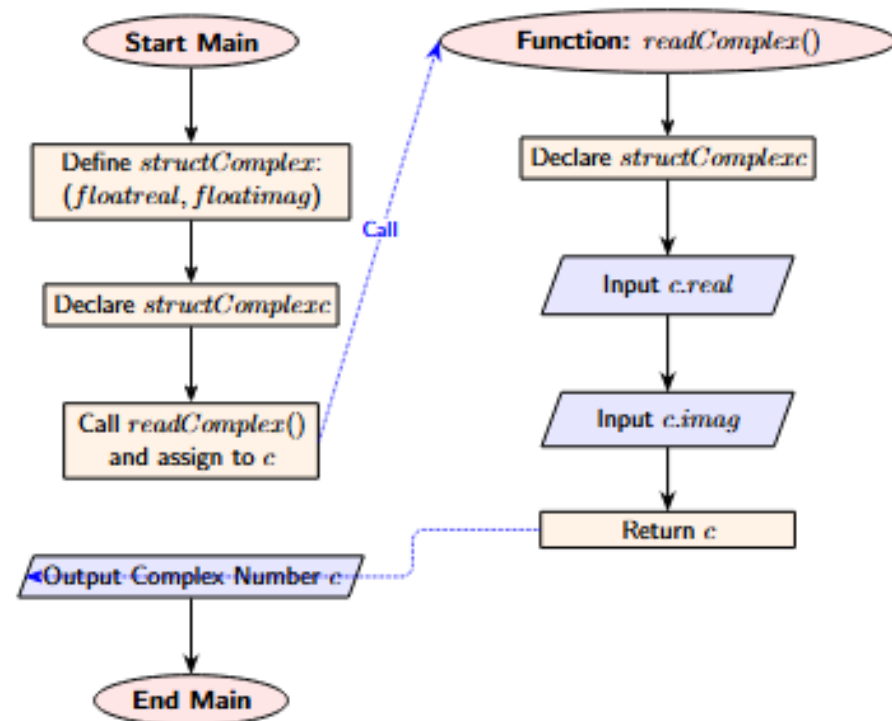
INPUT:

```
1  #include <stdio.h>
2  struct Complex {
3      float real;
4      float imag;
5  };
6  struct Complex readComplex() {
7      struct Complex c;
8      printf("Enter real part: ");
9      scanf("%f", &c.real);
10     printf("Enter imaginary part: ");
11     scanf("%f", &c.imag);
12     return c;
13 }
14 int main() {
15     struct Complex c;
16     printf("Reading a complex number:\n");
17     c = readComplex();
18     printf("\nYou entered: %.2f + %.2fi\n", c.real, c.imag);
19     return 0;
20 }
21
```

OUTPUT:

```
Reading a complex number:  
Enter real part: 45  
Enter imaginary part: : b  
  
You entered: 45.00 + 0.00i  
  
=== Code Execution Successful ===
```

FLOWCHART:



b. writing the complex part

Algorithm

1. Start
2. Declare a structure `Complex` with two members: `real` and `imag`
3. In `main()`, declare a variable `c` of type `struct Complex`
4. Ask the user to enter the real part of the complex number
5. Read and store the real part in `c.real`
6. Ask the user to enter the imaginary part of the complex number
7. Read and store the imaginary part in `c.imag`
8. Print the message: "The complex number is:"
9. Call the function `writeComplex(c)`
10. Inside `writeComplex()`
 1. If imaginary part is **positive or zero**, print `real + imag i`
 2. Else print `real - imag i` (imaginary part printed as positive value)
11. End

PSUEDOCODE:

```
STRUCT Complex
    real : float
    imag : float
END STRUCT

FUNCTION writeComplex(c : Complex)
    IF c.imag >= 0 THEN
        PRINT c.real, "+", c.imag, "i"
    ELSE
        PRINT c.real, "-", ABS(c.imag), "i"
    ENDIF
END FUNCTION

BEGIN
    DECLARE c AS Complex

    PRINT "Enter the real part: "
    INPUT c.real

    PRINT "Enter the imaginary part: "
    INPUT c.imag

    PRINT "The complex number is: "
    CALL writeComplex(c)

END
```

INPUT:

```
1  #include <stdio.h>
2  struct Complex {
3      float real;
4      float imag;
5  };
6  void writeComplex(struct Complex c) {
7      if (c.imag >= 0)
8          printf("%.2f + %.2fi\n", c.real, c.imag);
9      else
10         printf("%.2f - %.2fi\n", c.real, -c.imag);
11 }
12 int main() {
13     struct Complex c;
14     printf("Enter the real part: ");
15     scanf("%f", &c.real);
16     printf("Enter the imaginary part: ");
17     scanf("%f", &c.imag);
18     printf("\nThe complex number is: ");
19     writeComplex(c);
20     return 0;
21 }
```

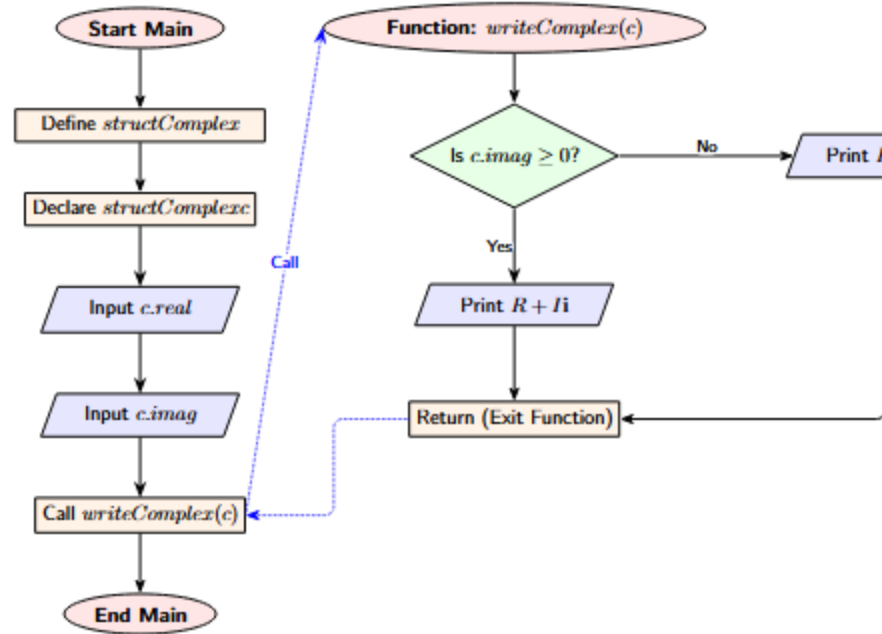
OUTPUT:

```
Enter the real part: 45
Enter the imaginary part: 6

The complex number is: 45.00 + 6.00i

=== Code Execution Successful ===
```


FLOWCHART:



c. Addition and Subtraction of two complex number

ALGORITHM:

1. **Start**
2. Declare a structure **Complex** with two members:
3. `real`
4. `imag`
5. Define function **add(a, b)** that:
6. Creates a Complex variable `r`
7. Computes `r.real = a.real + b.real`
8. Computes `r.imag = a.imag + b.imag`
9. Creates a Complex variable `r`
10. Computes `r.real = a.real - b.real`
11. Computes `r.imag = a.imag - b.imag`
12. Returns `r`
13. In **main**:
14. `y`:
15. Sum in form `a + bi`
16. Difference in form `a + bi`
17. **Stop**

PSUEDOCODE:

```
START

DECLARE c1, c2, sum, diff : Complex

PRINT "Enter real and imaginary part of first complex number:"
READ c1.real, c1.imag

PRINT "Enter real and imaginary part of second complex number:"
READ c2.real, c2.imag

sum ← add(c1, c2)
diff ← subtract(c1, c2)

PRINT "Sum = ", sum.real, " + ", sum.imag, "i"
PRINT "Difference = ", diff.real, " + ", diff.imag, "i"

END
```

INPUT:

```
1  #include <stdio.h>
2  struct Complex {
3      float real;
4      float imag;
5  };
6  struct Complex add(struct Complex a, struct Complex b) {
7      struct Complex r;
8      r.real = a.real + b.real;
9      r.imag = a.imag + b.imag;
10     return r;
11 }
12
13 struct Complex subtract(struct Complex a, struct Complex b) {
14     struct Complex r;
15     r.real = a.real - b.real;
16     r.imag = a.imag - b.imag;
17     return r;
18 }
19 int main() {
20     struct Complex c1, c2, sum, diff;
21     printf("Enter real and imaginary part of first complex number: ");
22     scanf("%f %f", &c1.real, &c1.imag);
23     printf("Enter real and imaginary part of second complex number: ");
24     scanf("%f %f", &c2.real, &c2.imag);
25     sum = add(c1, c2);
26     diff = subtract(c1, c2);
27     printf("\nSum = %.2f + %.2fi\n", sum.real, sum.imag);
28     printf("Difference = %.2f + %.2fi\n", diff.real, diff.imag);
29     return 0;
30 }
```

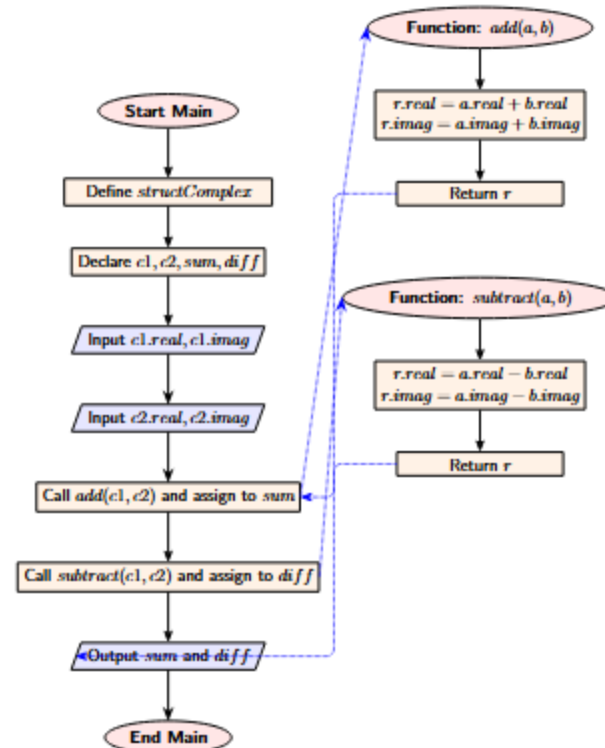
OUTPUT:

```
Enter real and imaginary part of first complex number: 5 6
Enter real and imaginary part of second complex number: 6 7

Sum = 11.00 + 13.00i
Difference = -1.00 + -1.00i

=== Code Execution Successful ===
```

FLOWCHART:



name, basic pay. The DA is computed as 52% of the basic pay. Gross-salary (basic pay + DA). Print the employees name and gross salary.

- **Algorithm**

- **Step 1:** Start

Step 2: Create a structure Employee with fields: name, basic, and gross.

Step 3: Declare an array emp[100] of type Employee.

Step 4: Display a message to enter employee details.

Step 5: Repeat steps for $i = 0$ to 99

- a. Read employee name
- b. Read basic salary
- c. Calculate $DA = 52\%$ of basic pay
- d. Compute gross salary = basic + DA
- e. Store values in structure array

Step 6: Print table heading "Employee Name" and "Gross Salary".

Step 7: Loop again from $i = 0$ to 99

Display each employee's name and gross salary

Step 8: Stop

PSUEDOCODE:

```
BEGIN

  DEFINE STRUCT Employee
    name : string
    basic : float
    gross : float
  END STRUCT

  DECLARE emp[100] AS ARRAY OF Employee

  PRINT "Enter details of 100 employees:"

  FOR i ← 0 TO 99 DO
    PRINT "Employee", i+1
    PRINT "Enter name: "
    READ emp[i].name

    PRINT "Enter basic pay: "
    READ emp[i].basic

    DA ← 0.52 * emp[i].basic
    emp[i].gross ← emp[i].basic + DA
  END FOR

  PRINT "-----"
  PRINT "Employee Name      Gross Salary"
  PRINT "-----"

  FOR i ← 0 TO 99 DO
    PRINT emp[i].name, emp[i].gross
  END FOR

END
```


INPUT:

```
1  #include <stdio.h>
2  struct Employee {
3      char name[50];
4      float basic;
5      float gross;
6  };
7  int main() {
8      struct Employee emp[100];
9      printf("Enter details of 100 employees:\n");
10     for (int i = 0; i < 100; i++) {
11         printf("\nEmployee %d\n", i + 1);
12         printf("Enter name: ");
13         scanf("%s", emp[i].name);
14         printf("Enter basic pay: ");
15         scanf("%f", &emp[i].basic);
16         float DA = 0.52 * emp[i].basic;
17         emp[i].gross = emp[i].basic + DA;
18     }
19     printf("\n-----\n");
20     printf("Employee Name\t\tGross Salary\n");
21     printf("-----\n");
22     for (int i = 0; i < 100; i++) {
23         printf("%-20s %.2f\n", emp[i].name, emp[i].gross);
24     }
25     return 0;
26 }
```

OUTPUT:

```
Enter details of 100 employees:
```

```
Employee 1
```

```
Enter name: A
```

```
Enter basic pay: 1
```

```
Employee 2
```

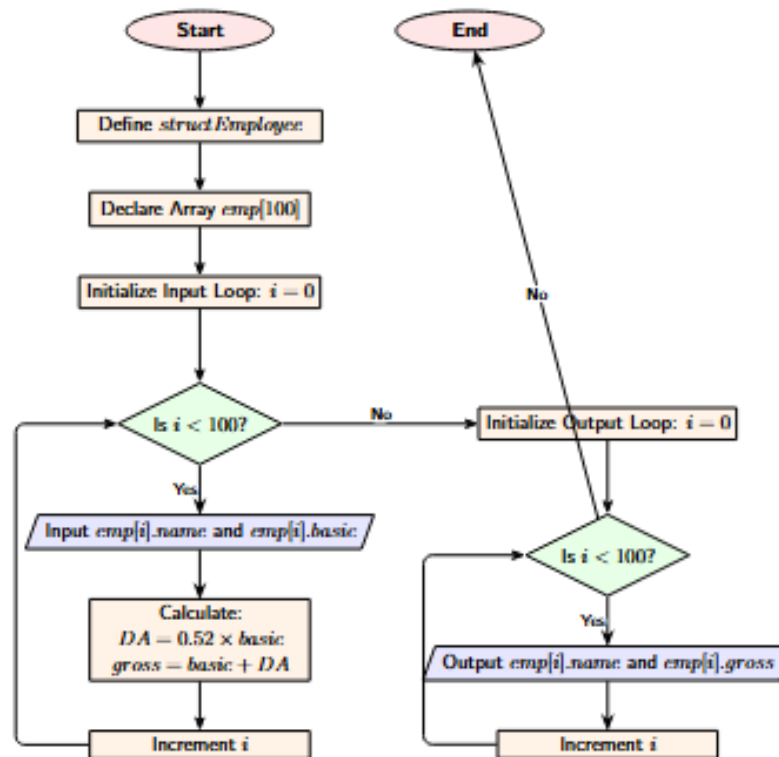
```
Enter name: B
```

```
Enter basic pay: 2
```

```
Employee 3
```

```
Enter name: |
```

FLOWCHART:



3. Create a Book structure containing book_id, title, author name and price. Write a C program to pass a structure as a function argument and print the book details.

Algorithm:

1. Start
2. Declare a structure **Book** with members:
 1. book_id
 2. title
 3. author
 4. price
3. Declare a variable **b** of type structure Book
4. Ask the user to enter book ID
5. Read book ID
6. Ask the user to enter book title
7. Read book title
8. Ask the user to enter author name
9. Read author name
10. Ask the user to enter price
11. Read price
12. Call function `printBook(b)`
13. Inside `printBook(b)`
 1. Print book ID
 2. Print title
 3. Print author
 4. Print price

PSUEDOCODE:

```
BEGIN

DEFINE STRUCT Book
    INTEGER book_id
    STRING title
    STRING author
    FLOAT price
END STRUCT

FUNCTION printBook(b)
    PRINT "--- Book Details ---"
    PRINT "Book ID :", b.book_id
    PRINT "Title :", b.title
    PRINT "Author :", b.author
    PRINT "Price :", b.price
END FUNCTION

DECLARE b AS Book

PRINT "Enter Book ID: "
READ b.book_id

PRINT "Enter Book Title: "
READ b.title

PRINT "Enter Author Name: "
READ b.author

PRINT "Enter Price: "
READ b.price

CALL printBook(b)

END
```

INPUT:

```
1  #include <stdio.h>
2  struct Book {
3      int book_id;
4      char title[50];
5      char author[50];
6      float price;
7  };
8  void printBook(struct Book b) {
9      printf("\n--- Book Details ---\n");
10     printf("Book ID   : %d\n", b.book_id);
11     printf("Title     : %s\n", b.title);
12     printf("Author    : %s\n", b.author);
13     printf("Price      : %.2f\n", b.price);
14 }
15 int main() {
16     struct Book b;
17     printf("Enter Book ID: ");
18     scanf("%d", &b.book_id);
19     getchar();
20     printf("Enter Book Title: ");
21     fgets(b.title, sizeof(b.title), stdin);
22     printf("Enter Author Name: ");
23     fgets(b.author, sizeof(b.author), stdin);
24     printf("Enter Price: ");
25     scanf("%f", &b.price);
26     printBook(b);
27     return 0;
28 }
29
```

OUTPUT:

```
Enter Book ID: 12
Enter Book Title: Jungle book
Enter Author Name: Siddhant
Enter Price: 500

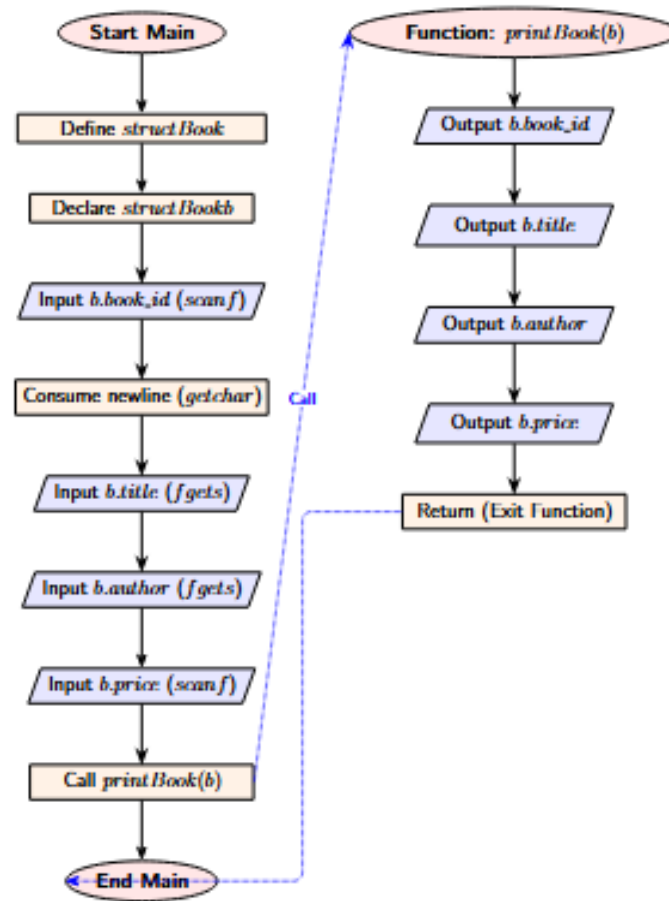
--- Book Details ---
Book ID   : 12
Title     : Jungle book

Author    : Siddhant

Price     : 500.00

=== Code Execution Successful ===
```

FLOWCHART:



4. Create a union containing 6 strings: name, home_address, hostel_address, city, state and zip. Write a C program to display your present address.

Algorithm

1. Start
2. Declare a union named **Address** containing:
 1. name
 2. home_address
 3. hostel_address
 4. city
 5. state
 6. zip
3. Declare a union variable **A**
4. Store the hostel address into **A.hostel_address** using string copy
5. Print "**Present Address:**"
6. Display the value stored in **A.hostel_address**

PSUEDOCODE:

```
BEGIN

DECLARE UNION Address
    STRING name
    STRING home_address
    STRING hostel_address
    STRING city
    STRING state
    STRING zip
END UNION

DECLARE variable A of type Address

COPY "ABC Hostel, Room 204, Campus Road" INTO A.hostel_address

PRINT "Present Address:"
PRINT A.hostel_address

END
```

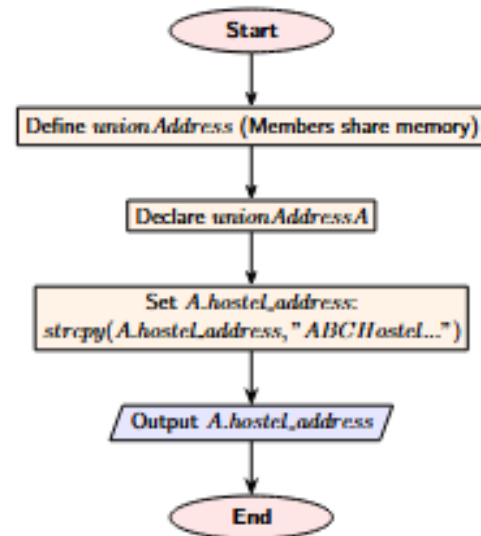
INPUT:

```
1  #include <stdio.h>
2  #include <string.h>
3  union Address {
4      char name[50];
5      char home_address[100];
6      char hostel_address[100];
7      char city[50];
8      char state[50];
9      char zip[10];
10 };
11 int main() {
12     union Address A;
13     strcpy(A.hostel_address, "ABC Hostel, Room 204, Campus Road");
14     printf("Present Address:\n");
15     printf("%s\n", A.hostel_address);
16     return 0;
17 }
```

OUTPUT:

```
Present Address:  
ABC Hostel, Room 204, Campus Road  
  
=== Code Execution Successful ===
```

FLOWCHART:



Experiment 8: Pointers

1. Declare different types of pointers (int, float, char) and initialize them with the addresses of variables. Print the values of both the pointers and the variables they point to.

Algorithm

1. Start
2. Declare integer variable `a = 10`
3. Declare float variable `b = 12.5`
4. Declare char variable `c = 'X'`
5. Declare pointer `p1` and store the address of `a`
6. Declare pointer `p2` and store the address of `b`
7. Declare pointer `p3` and store the address of `c`
8. Print value of `a`, address in `p1`, and value pointed by `p1`
9. Print value of `b`, address in `p2`, and value pointed by `p2`
10. Print value of `c`, address in `p3`, and value pointed by `p3`
11. End

PSUEDOCODE:

```
BEGIN
```

```
SET a = 10
```

```
SET b = 12.5
```

```
SET c = 'X'
```

```
SET p1 = address of a
```

```
SET p2 = address of b
```

```
SET p3 = address of c
```

```
PRINT "Value of a", a
```

```
PRINT "Address stored in p1", p1
```

```
PRINT "Value pointed by p1", *p1
```

```
PRINT "Value of b", b
```

```
PRINT "Address stored in p2", p2
```

```
PRINT "Value pointed by p2", *p2
```

```
PRINT "Value of c", c
```

```
PRINT "Address stored in p3", p3
```

```
PRINT "Value pointed by p3", *p3
```

```
END
```

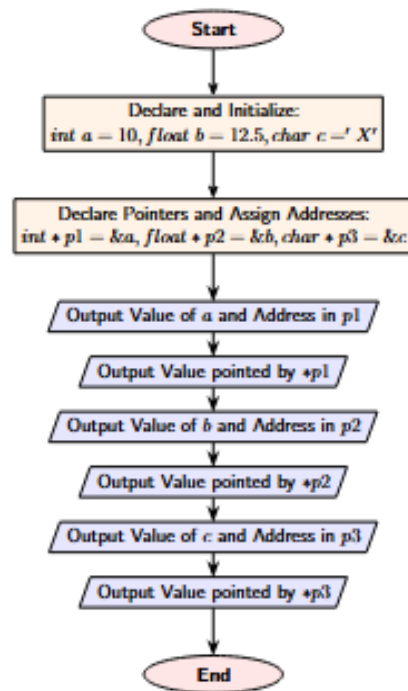
INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int a = 10;
4      float b = 12.5;
5      char c = 'X';
6      int *p1 = &a;
7      float *p2 = &b;
8      char *p3 = &c;
9      printf("Value of a = %d\n", a);
10     printf("Address stored in int pointer p1 = %p\n", (void*)p1);
11     printf("Value pointed by p1 = %d\n\n", *p1);
12     printf("Value of b = %.2f\n", b);
13     printf("Address stored in float pointer p2 = %p\n", (void*)p2);
14     printf("Value pointed by p2 = %.2f\n\n", *p2);
15     printf("Value of c = %c\n", c);
16     printf("Address stored in char pointer p3 = %p\n", (void*)p3);
17     printf("Value pointed by p3 = %c\n", *p3);
18     return 0;
19 }
```


OUTPUT:

```
Value of a = 10  
Address stored in int pointer p1 = 0x7ffd761c1f14  
Value pointed by p1 = 10  
  
Value of b = 12.50  
Address stored in float pointer p2 = 0x7ffd761c1f10  
Value pointed by p2 = 12.50  
  
Value of c = X  
Address stored in char pointer p3 = 0x7ffd761c1f0f  
Value pointed by p3 = X  
  
=== Code Execution Successful ===
```

FLOWCHART:



2. Perform pointer arithmetic (increment and decrement) on pointers of different data types. Observe how the memory addresses change and the effects on data access.

Algorithm

1. Start
2. Declare integer variable a = 10
3. Declare float variable b = 20.5
4. Declare char variable c = 'X'
5. Declare pointers:
 1. p1 pointing to a
 2. p2 pointing to b
 3. p3 pointing to c
6. Print initial pointer addresses (p1, p2, p3)
7. Increment pointers:
 1. p1++
 2. p2++
 3. p3++
8. Print the pointer addresses after increment
9. Decrement pointers:
 1. p1--
 2. p2--
 3. p3--
10. Print pointer addresses after decrement
11. End

PSUEDOCODE:

```
BEGIN

SET a = 10
SET b = 20.5
SET c = 'X'

SET p1 = address of a
SET p2 = address of b
SET p3 = address of c

PRINT "Initial Addresses"
PRINT p1, p2, p3

INCREMENT p1
INCREMENT p2
INCREMENT p3

PRINT "Addresses After Increment"
PRINT p1, p2, p3

DECREMENT p1
DECREMENT p2
DECREMENT p3

PRINT "Addresses After Decrement"
PRINT p1, p2, p3

END
```

INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int a = 10;
4      float b = 20.5;
5      char c = 'X';
6      int *p1 = &a;
7      float *p2 = &b;
8      char *p3 = &c;
9      printf("Initial Addresses:\n");
10     printf("p1 (int*) = %p\n", (void*)p1);
11     printf("p2 (float*) = %p\n", (void*)p2);
12     printf("p3 (char*) = %p\n\n", (void*)p3);
13     p1++;
14     p2++;
15     p3++;
16     printf("Addresses After Increment:\n");
17     printf("p1 (int*) = %p (int pointer increases by %zu bytes)\n",
18           (void*)p1, sizeof(int));
19     printf("p2 (float*) = %p (float pointer increases by %zu bytes)\n",
20           (void*)p2, sizeof(float));
21     printf("p3 (char*) = %p (char pointer increases by %zu bytes)\n\n",
22           (void*)p3, sizeof(char));
23     p1--;
24     p2--;
25     p3--;
26     printf("Addresses After Decrement (back to original):\n");
27     printf("p1 (int*) = %p\n", (void*)p1);
28     printf("p2 (float*) = %p\n", (void*)p2);
29     printf("p3 (char*) = %p\n", (void*)p3);
30     return 0;
```

OUTPUT:

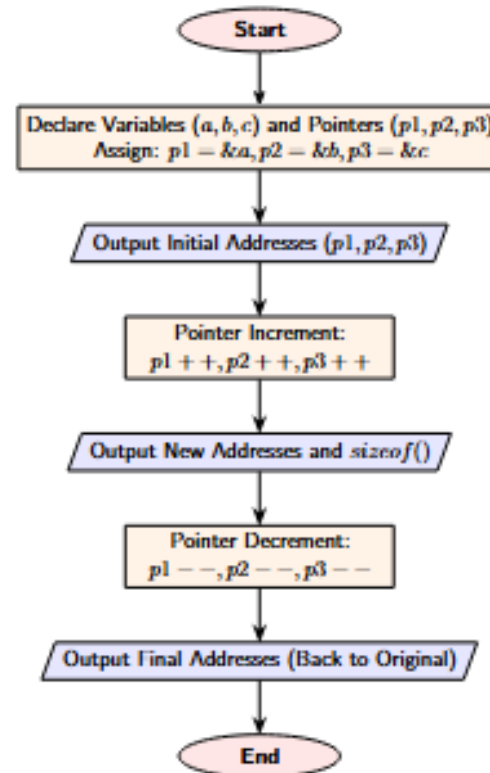
```
Initial Addresses:
p1 (int*) = 0x7fffcbc79704
p2 (float*)= 0x7fffcbc79700
p3 (char*) = 0x7fffcbc796ff

Addresses After Increment:
p1 (int*) = 0x7fffcbc79708 (int pointer increases by 4 bytes)
p2 (float*)= 0x7fffcbc79704 (float pointer increases by 4 bytes)
p3 (char*) = 0x7fffcbc79700 (char pointer increases by 1 bytes)

Addresses After Decrement (back to original):
p1 (int*) = 0x7fffcbc79704
p2 (float*)= 0x7fffcbc79700
p3 (char*) = 0x7fffcbc796ff

=== Code Execution Successful ===
```

FLOWCHART:



3. Write a function that accepts pointers as parameters. Pass variables by reference using pointers and modify their values within the function.

ALGORITHM:

1. Start
2. Declare two integer variables: x and y
3. Ask the user to enter two numbers
4. Read the values of x and y
5. Call the function `changeValues(&x, &y)`
6. Inside the function:
7. Add 10 to the value pointed by pointer a
8. Multiply the value pointed by pointer b by 2
9. Return to main
10. Print the updated values of x and y
11. Stop

PSUEDOCODE:

```
BEGIN

    DECLARE x, y AS INTEGER

    PRINT "Enter two numbers: "
    INPUT x, y

    CALL changeValues(address of x, address of y)

    PRINT "Updated values: ", x, y

END

FUNCTION changeValues(a, b as POINTER TO INTEGER)

    SET value_at(a) = value_at(a) + 10
    SET value_at(b) = value_at(b) * 2

END FUNCTION
```

INPUT:

```
1  #include <stdio.h>
2  void changeValues(int *a, int *b) {
3      *a = *a + 10;
4      *b = *b * 2;
5  }
6  int main() {
7      int x, y;
8      printf("Enter two numbers: ");
9      scanf("%d %d", &x, &y);
10     changeValues(&x, &y);
11     printf("Updated values: %d %d\n", x, y);
12     return 0;
13 }
14
```

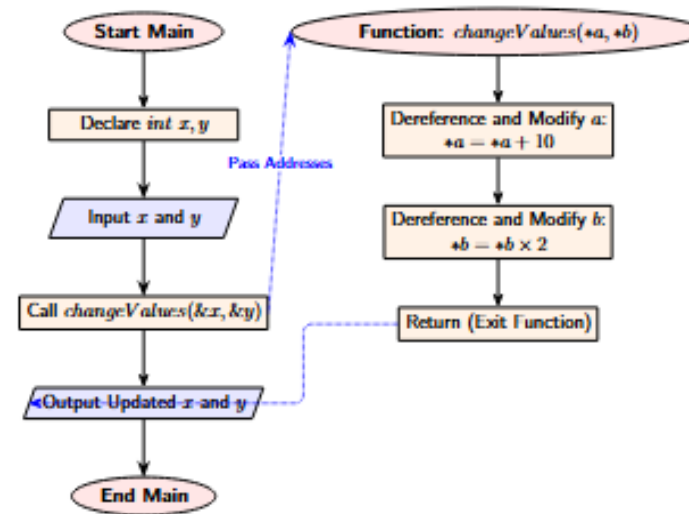
OUTPUT:

```
Enter two numbers: 4 5
```

```
Updated values: 14 10
```

```
=== Code Execution Successful ===
```

FLOWCHART:



Experiment 9: File handling

1. Write a program to create a new file and write text into it

ALGORITHM

1. START

2. DECLARE A FILE POINTER FP.

3. OPEN A FILE NAMED "EXAMPLE.TXT" IN **WRITE MODE** USING
FP = FOPEN("EXAMPLE.TXT", "W").

4. SINCE NO WRITING IS DONE, SIMPLY PROCEED TO CLOSE THE
FILE.

5. CLOSE THE FILE USING FCLOSE(FP).

6. END THE PROGRAM.

PSUEDOCODE:

```
START

DECLARE file pointer fp

fp ← OPEN file "example.txt" in write mode

CLOSE file fp

END
```

INPUT:

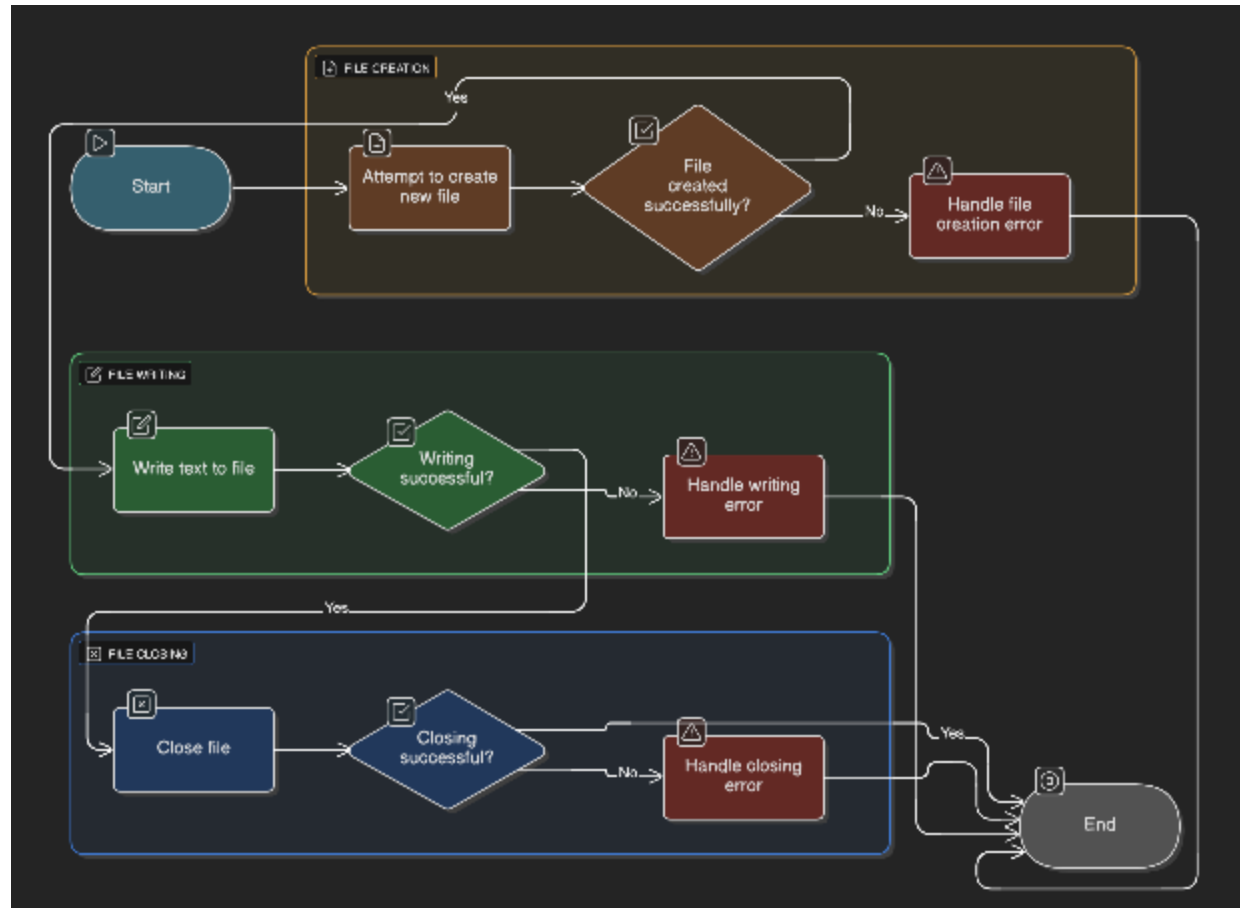
```
1  #include <stdio.h>
2  int main() {
3      FILE *fp = fopen("example.txt", "w");
4      fclose(fp);
5      return 0;
6  }
7
```

OUTPUT:

```
Segmentation fault
```

```
=== Code Exited With Errors ===
```


FLOWCHART:



2. Open an existing file and read its content character by character, and then close the file.

Algorithm

1. Start
2. Declare a file pointer `fp` and a character variable `ch`.
3. Open the file "example.txt" in **read mode** using `fopen("example.txt", "r")`.
4. If the file does not exist (`fp == NULL`):
 1. Print "File not found!"
 2. Exit the program
5. Print a message: **"Reading file character by character:"**
6. Read a character using `fgetc(fp)` and store it in `ch`.
7. While `ch` is not equal to EOF:
 1. Display the character using `putchar(ch)`
 2. Read the next character
8. After reaching EOF, close the file using `fclose(fp)`.
9. End the program.

PSUEDOCODE:

```
START

DECLARE file pointer fp
DECLARE character ch

fp ← OPEN "example.txt" in read mode

IF fp == NULL THEN
    PRINT "File not found!"
    TERMINATE PROGRAM
ENDIF

PRINT "Reading file character by character:"

ch ← fgetc(fp)

WHILE ch != EOF DO
    PRINT character ch
    ch ← fgetc(fp)
END WHILE

CLOSE file fp

END
```

INPUT:

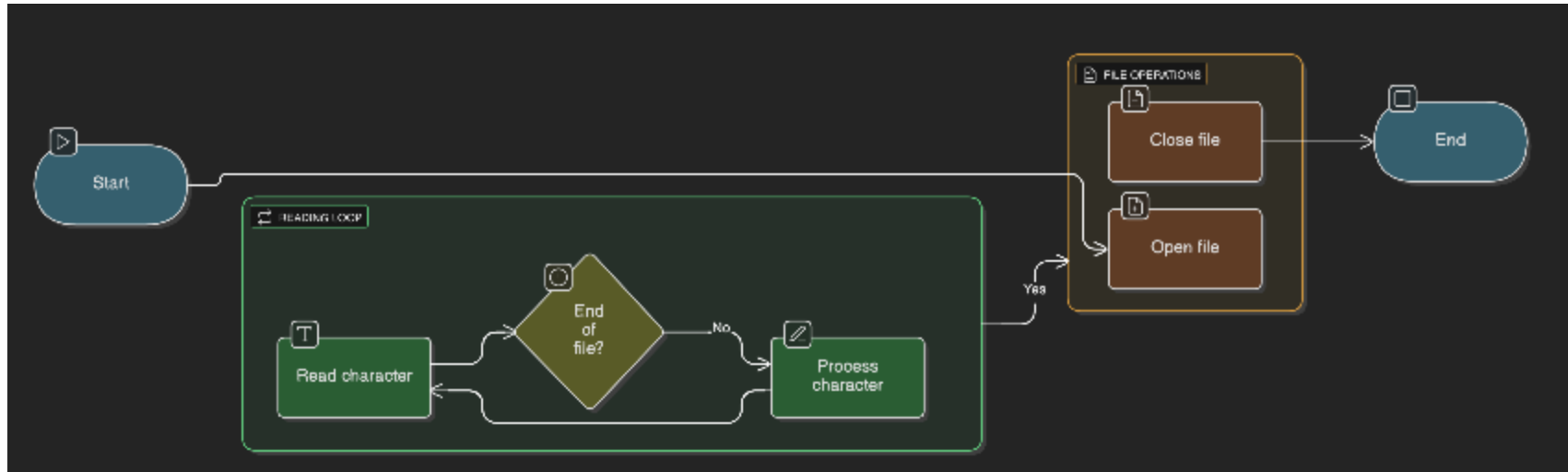
```
1 #include <stdio.h>
2 int main() {
3     FILE *fp;
4     char ch;
5     fp = fopen("example.txt", "r");
6     if (fp == NULL) {
7         printf("File not found!\n");
8         return 1;
9     }
10    printf("Reading file character by character:\n");
11    while ((ch = fgetc(fp)) != EOF) {
12        putchar(ch);
13    }
14    fclose(fp);
15    return 0;
16 }
17
```

OUTPUT:

```
Segmentation fault
```

```
=== Code Exited With Errors ===
```

FLOWCHART:



3. Write a function that accepts pointers as parameters. Pass variables by reference using pointers and modify their values within the function

Algorithm

1. Start
2. Declare a file pointer `fp` and a string array `line[200]`.
3. Open the file "example.txt" in read mode using
`fp = fopen("example.txt", "r").`
4. If `fp == NULL`, print **"File not found!"** and terminate the program.
5. Print **"Reading file line by line:"**
6. Use a loop that continues while
`fgets(line, sizeof(line), fp) != NULL`
 1. Inside the loop, print the line using `printf("%s", line).`
7. After finishing reading, close the file with `fclose(fp).`
8. End the program.

PSUEDOCODE:

```
START

DECLARE file pointer fp
DECLARE string line[200]

fp ← OPEN "example.txt" in read mode

IF fp == NULL THEN
    PRINT "File not found!"
    EXIT PROGRAM
ENDIF

PRINT "Reading file line by line:"

WHILE fgets(line, 200, fp) is NOT NULL DO
    PRINT line
END WHILE

CLOSE file fp

END
```


INPUT:

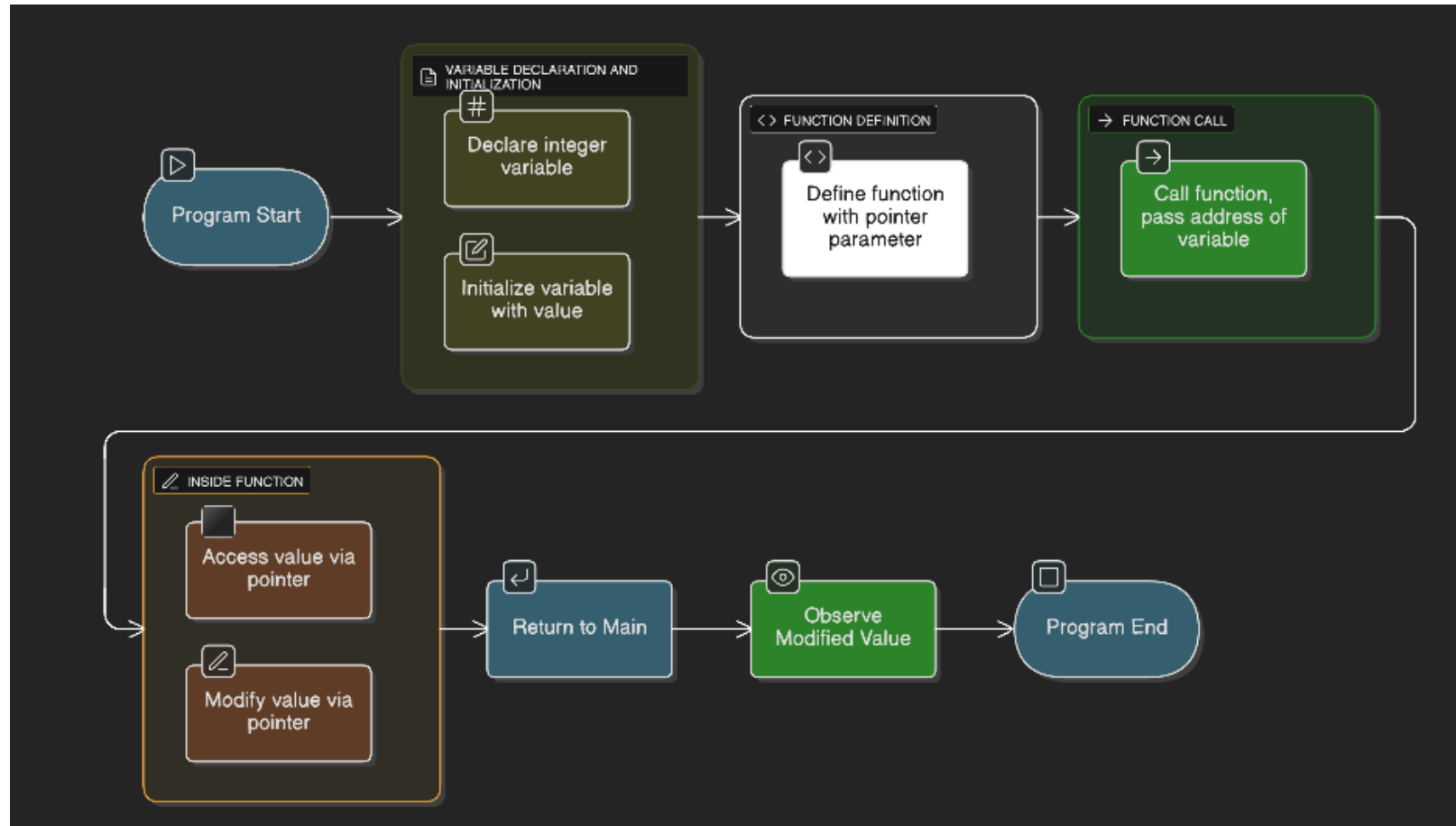
```
1  #include <stdio.h>
2  int main() {
3      FILE *fp;
4      char line[200];
5      fp = fopen("example.txt", "r");
6      if (fp == NULL) {
7          printf("File not found!\n");
8          return 1;
9      }
10     printf("Reading file line by line:\n");
11     while (fgets(line, sizeof(line), fp) != NULL) {
12         printf("%s", line);
13     }
14     fclose(fp);
15     return 0;
16 }
17
```

OUTPUT:

```
File not found!
```

```
=== Code Exited With Errors ===
```

FLOWCHART:



Experiment 10: Dynamic Memory Allocation

1: Create a Simple Linked List in C Using Pointer and Structure

Algorithm

1. Start
2. Define a structure `Node` with `data` and pointer `next`.
3. Declare three node pointers: `head`, `second`, `third`.
4. Allocate memory dynamically for all three nodes using `malloc`.
5. Assign data to each node.
6. Link the nodes using `next` pointer:
 1. `head` → `second`
 2. `second` → `third`
 3. `third` → `NULL`
7. Traverse the list and print all node values.
8. Stop.

PSEUDOCODE:

```
START

DEFINE structure Node (data, next pointer)

DECLARE head, second, third as Node pointers

ALLOCATE memory for head, second, third

SET head.data = 10
SET head.next = second

SET second.data = 20
SET second.next = third

SET third.data = 30
SET third.next = NULL

SET temp = head

WHILE temp IS NOT NULL DO
    PRINT temp.data
    temp = temp.next
END WHILE

END
```

INPUT:

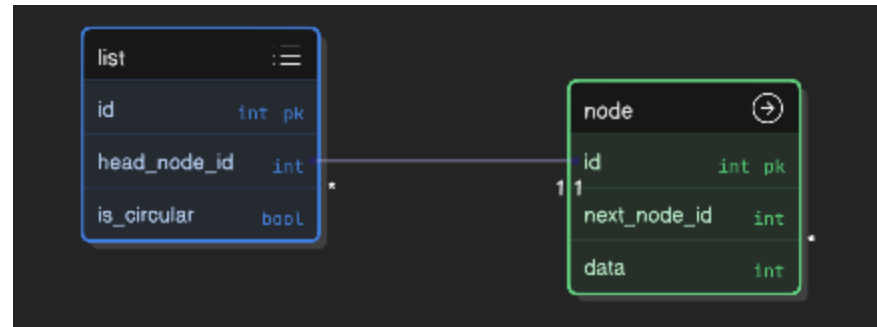
```
1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node *next;
6  };
7  int main() {
8      struct Node *head = NULL, *second = NULL, *third = NULL;
9      head = (struct Node *)malloc(sizeof(struct Node));
10     second = (struct Node *)malloc(sizeof(struct Node));
11     third = (struct Node *)malloc(sizeof(struct Node));
12     head->data = 10;
13     head->next = second;
14     second->data = 20;
15     second->next = third;
16     third->data = 30;
17     third->next = NULL;
18     struct Node *temp = head;
19     printf("Linked List: ");
20     while (temp != NULL) {
21         printf("%d -> ", temp->data);
22         temp = temp->next;
23     }
24     printf("NULL\n");
25
26     return 0;
27 }
28
```

OUTPUT:

```
Linked List: 10 -> 20 -> 30 -> NULL
```

```
=== Code Execution Successful ===
```

FLOWCHART:



2: Insert an Item in the Middle of a Linked List

Algorithm

1. Start
2. Create a linked list with at least 3 nodes.
3. Define the function `insertMiddle()`.
4. Create a new node using `malloc`.
5. Use two pointers (`slow`, `fast`) to find the middle of the list.
 1. `slow` moves one step
 2. `fast` moves two steps
6. When `fast` reaches the end, `slow` is at the middle node.
7. Insert the new node after `slow`.
8. Update pointers:
 1. `newNode` $\rightarrow$ `slow->next`
 2. `slow->next` $\rightarrow$ `newNode`
9. Traverse the list and print updated data.
10. Stop.

PSUEDOCODE:

```
START

CREATE linked list with head → second → third

FUNCTION insertMiddle(head, newValue):
    CREATE newNode
    newNode.data = newValue

    SET slow = head
    SET fast = head.next

    WHILE fast != NULL AND fast.next != NULL DO
        slow = slow.next
        fast = fast.next.next
    END WHILE

    newNode.next = slow.next
    slow.next = newNode
END FUNCTION

CALL insertMiddle(head, 99)

TRAVERSE the list and PRINT all values

END
```

INPUT:

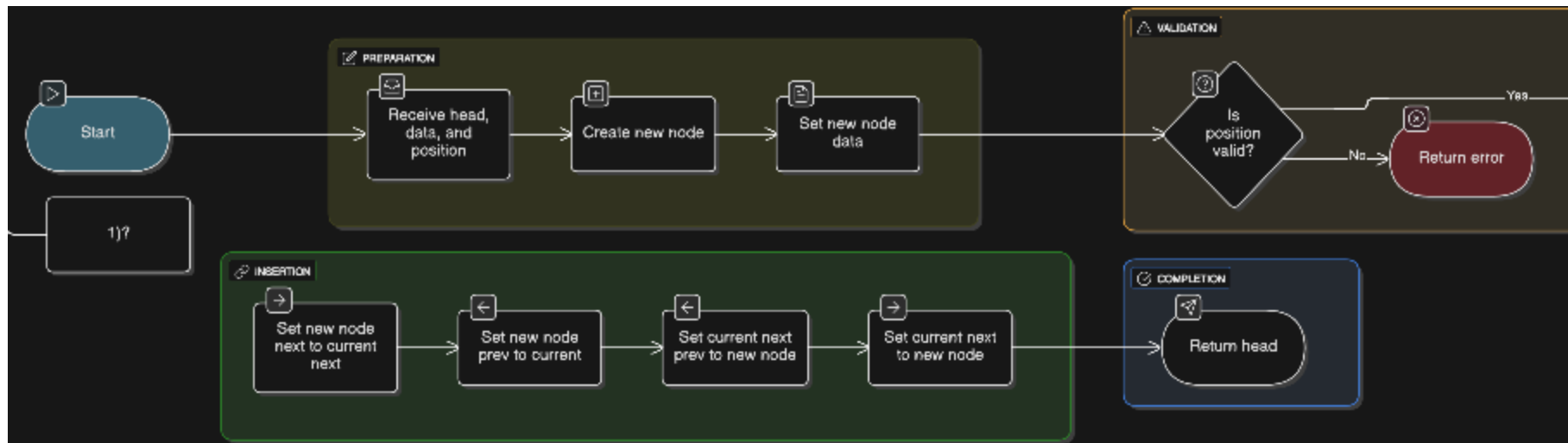
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Node {
4     int data;
5     struct Node *next;
6 };
7 void insertMiddle(struct Node *head, int newValue) {
8     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
9     newNode->data = newValue;
10    struct Node *slow = head;
11    struct Node *fast = head->next;
12    while (fast != NULL && fast->next != NULL) {
13        slow = slow->next;
14        fast = fast->next->next;
15    }
16    newNode->next = slow->next;
17    slow->next = newNode;
18 }
19 int main() {
20     struct Node *head = NULL, *second = NULL, *third = NULL;
21     head = malloc(sizeof(struct Node));
22     second = malloc(sizeof(struct Node));
23     third = malloc(sizeof(struct Node));
24
25     head->data = 10; head->next = second;
26     second->data = 20; second->next = third;
27     third->data = 30; third->next = NULL;
28     insertMiddle(head, 99);
29     struct Node *temp = head;
30     printf("Updated Linked List: ");
31     while (temp != NULL) {
32         printf("%d -> ", temp->data);
33         temp = temp->next;
34     }
35     printf("NULL\n");
36     return 0;
37 }
38
```

OUTPUT:

```
Updated Linked List: 10 -> 20 -> 99 -> 30 -> NULL
```

```
=== Code Execution Successful ===
```

FLOWCHART:



Experiment 11: Bitwise Operator

1. Program to Apply Bitwise OR, AND, NOT Operators

Algorithm

1. Start
2. Declare two integers a and b.
3. Initialize values ($a = 5, b = 3$).
4. Apply bitwise AND ($a \& b$) and display result.
5. Apply bitwise OR ($a \mid b$) and display result.
6. Apply bitwise NOT ($\sim a$) and display result.
7. Stop.

PSUEDOCODE:

```
START
```

```
SET a = 5
```

```
SET b = 3
```

```
PRINT a, b
```

```
PRINT a AND b
```

```
PRINT a OR b
```

```
PRINT NOT a
```

```
END
```

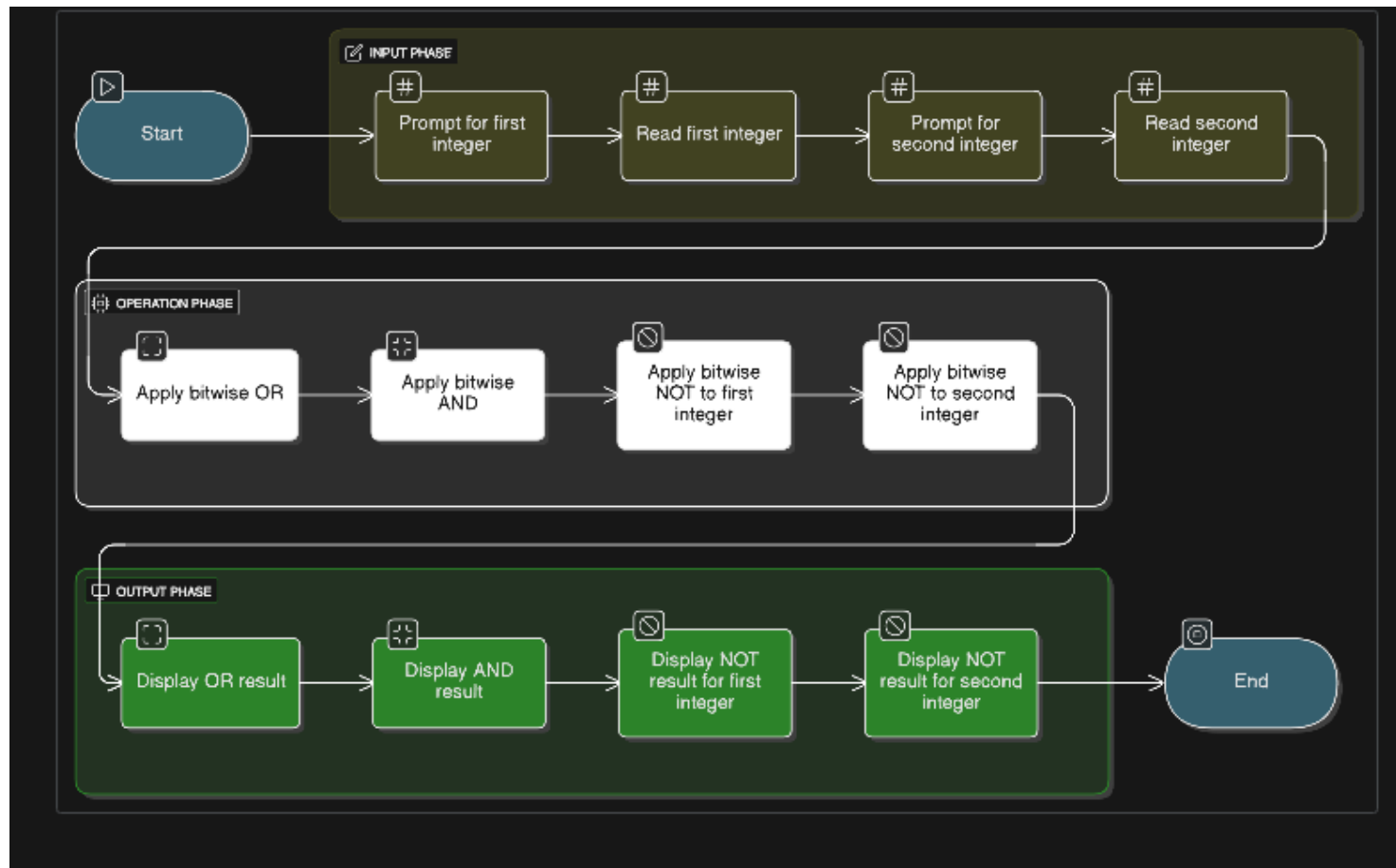
INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int a = 5;    // 0101 in binary
4      int b = 3;    // 0011 in binary
5      printf("a = %d, b = %d\n", a, b);
6      printf("a AND b = %d\n", a & b);
7      printf("a OR b  = %d\n", a | b);
8      printf("NOT a   = %d\n", ~a);
9      return 0;
10 }
11
```


OUTPUT:

```
a = 5, b = 3  
a AND b = 1  
a OR b  = 7  
NOT a   = -6  
  
=== Code Execution Successful ===
```

FLOWCHART:



2. Program to Apply Left Shift and Right Shift Operators

Algorithm

1. Start
2. Declare integer num and assign a value.
3. Display the original value.
4. Apply left shift (`num << 1`) and print the result.
5. Apply right shift (`num >> 1`) and print the result.
6. Stop.

PSEUDOCODE:

```
START

SET num = 8

PRINT "Original number:", num

PRINT "Left shift:", num << 1
PRINT "Right shift:", num >> 1

END
```

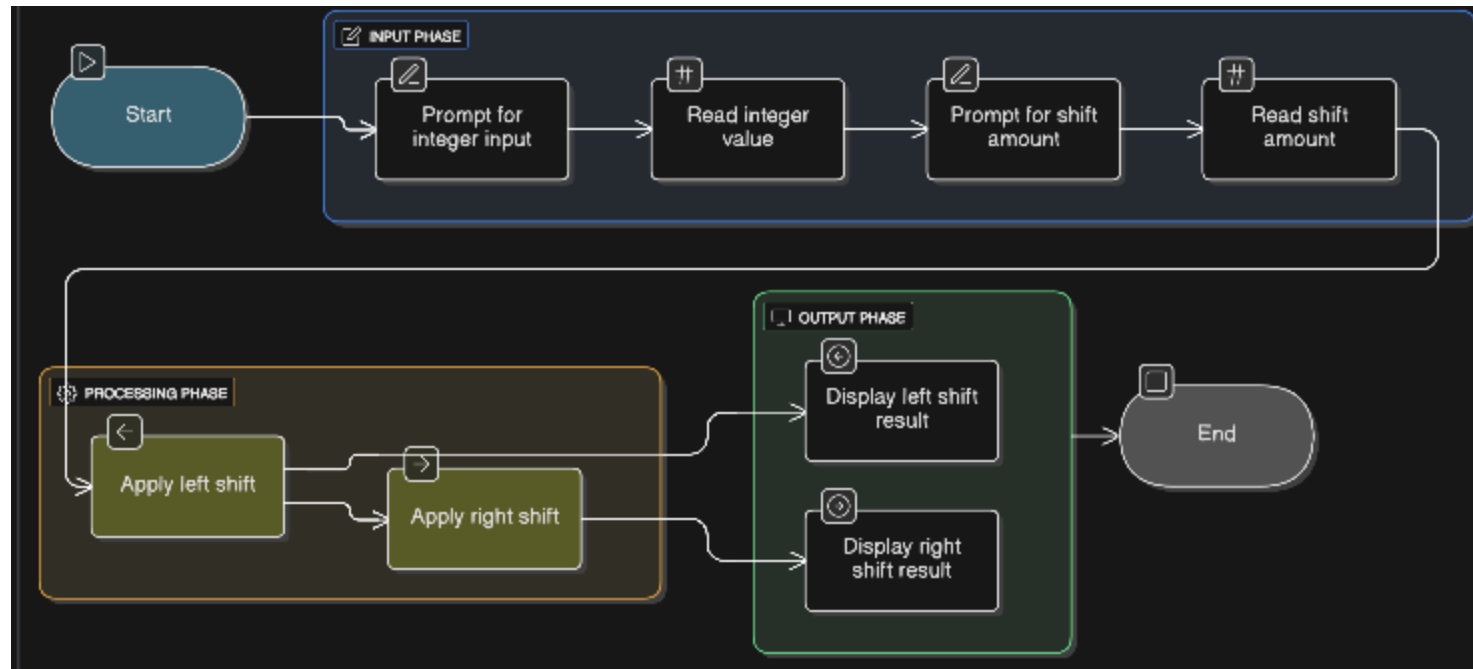
INPUT:

```
1  #include <stdio.h>
2  int main() {
3      int num = 8;    // 1000 in binary
4      printf("Original number: %d\n", num);
5      printf("Left shift by 1: %d\n", num << 1);    // multiply by 2
6      printf("Right shift by 1: %d\n", num >> 1);    // divide by 2
7      return 0;
8  }
9
```

OUTPUT:

```
Original number: 8  
Left shift by 1: 16  
Right shift by 1: 4  
  
=== Code Execution Successful ===
```

FLOWCHART:



Experiment 12: Preprocessor and Directives in C

1. Program to Define Constant Variables Using Preprocessor

Algorithm

1. Start
2. Use `#define` to create two constants: `PI` and `MAX`.
3. In `main()`, print the values of these constants.
4. Stop.

PSUEDOCODE:

```
START
```

```
DEFINE PI = 3.14
```

```
DEFINE MAX = 100
```

```
PRINT PI
```

```
PRINT MAX
```

```
END
```

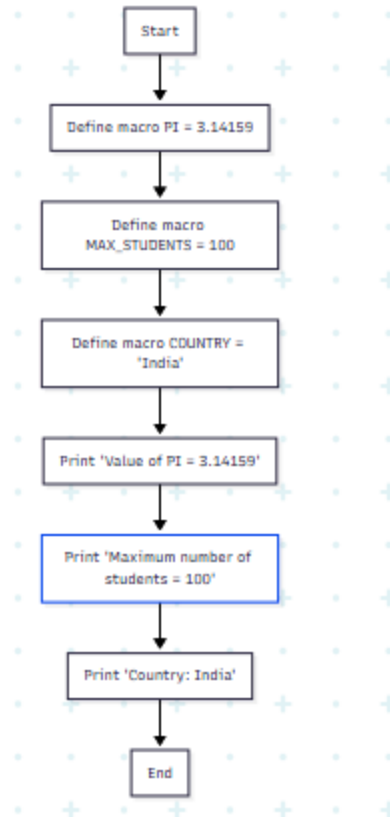
INPUT:

```
1  #include <stdio.h>
2  #define PI 3.14
3  #define MAX 100
4  int main() {
5      printf("Value of PI = %.2f\n", PI);
6      printf("Maximum allowed value = %d\n", MAX);
7      return 0;
8  }
9
```

OUTPUT:

```
Value of PI = 3.14  
Maximum allowed value = 100  
  
=== Code Execution Successful ===
```

FLOWCHART:



2. Program to Define a Function Using Preprocessor Macro

Algorithm

1. Start
2. Define a macro `SQUARE(x)` that returns $(x * x)$.
3. Declare an integer variable `n` with value 5.
4. Print the number.
5. Use the macro `SQUARE(n)` to compute square.
6. Display the result.
7. Stop.

PSUEDOCODE:

```
START

DEFINE macro SQUARE(x) = (x * x)

SET n = 5

PRINT n
PRINT SQUARE(n)

END
```

INPUT:

```
1  #include <stdio.h>
2  #define SQUARE(x) ((x) * (x))
3  int main() {
4      int n = 5;
5      printf("Number: %d\n", n);
6      printf("Square: %d\n", SQUARE(n));
7      return 0;
8  }
```

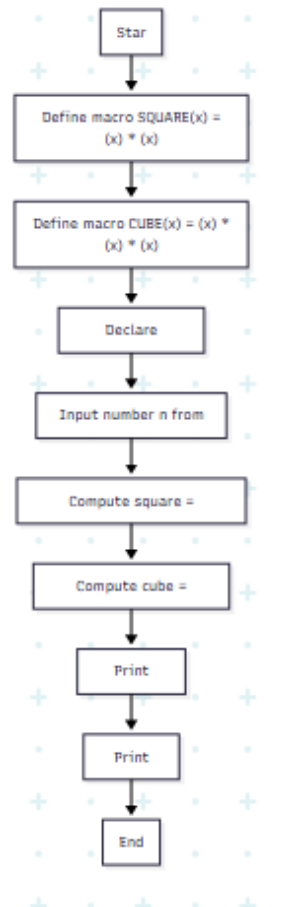
OUTPUT:

```
Number: 5
```

```
Square: 25
```

```
=== Code Execution Successful ===
```


FLOWCHART:



Experiment 13: Macros in C

1. Write a program to define multiple macros to perform arithmetic functions.

Algorithm

1. Start
2. Define four macros using `#define`:
 1. `ADD(a,b) → a + b`
 2. `SUB(a,b) → a - b`
 3. `MUL(a,b) → a * b`
 4. `DIV(a,b) → a / b` (handle divide-by-zero if needed)
3. In `main`, declare two integers `x` and `y`.
4. Read values of `x` and `y` from user.
5. Calculate sum using `ADD(x,y)` and print.
6. Calculate difference using `SUB(x,y)` and print.
7. Calculate product using `MUL(x,y)` and print.
8. Calculate division using `DIV(x,y)` and print.
9. STOP

PSUEDOCODE:

```
START

DEFINE macro ADD(a,b) = (a + b)
DEFINE macro SUB(a,b) = (a - b)
DEFINE macro MUL(a,b) = (a * b)
DEFINE macro DIV(a,b) = ( (b != 0) ? (a / b) : 0 )

DECLARE x, y

PRINT "Enter two integers:"
READ x, y

PRINT "ADD =", ADD(x, y)
PRINT "SUB =", SUB(x, y)
PRINT "MUL =", MUL(x, y)
PRINT "DIV =", DIV(x, y)

END
```

INPUT:

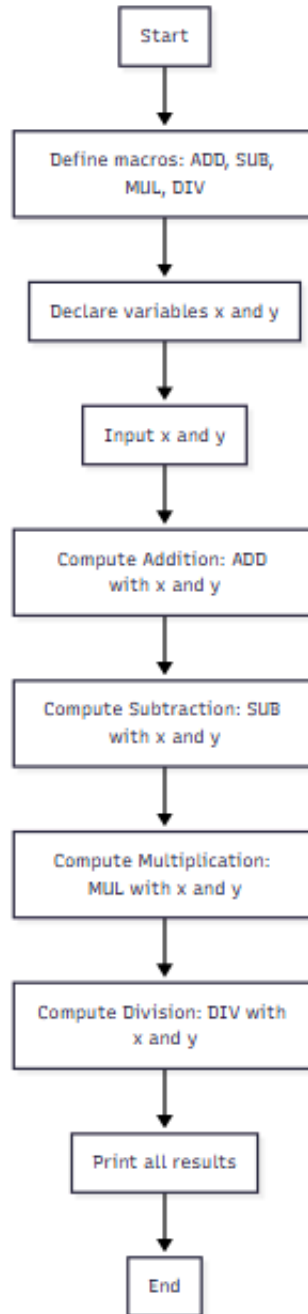
```
1  #include <stdio.h>
2  #define ADD(a,b)  ((a) + (b))
3  #define SUB(a,b)  ((a) - (b))
4  #define MUL(a,b)  ((a) * (b))
5  #define DIV(a,b)  ((b) != 0 ? (float)(a) / (b) : 0.0f)
6  int main(void) {
7      int x, y;
8      printf("Enter two integers: ");
9      scanf("%d %d", &x, &y);
10     printf("ADD  = %d\n", ADD(x, y));
11     printf("SUB  = %d\n", SUB(x, y));
12     printf("MUL  = %d\n", MUL(x, y));
13     printf("DIV  = %.2f\n", DIV(x, y));
14     return 0;
15 }
16
```

OUTPUT:

```
Enter two integers: 2 3
ADD  = 5
SUB  = -1
MUL  = 6
DIV  = 0.67

=== Code Execution Successful ===
```

FLOWCHART:



Experiment 14: Static Library in C

14.1 – Program to create a static library for arithmetic functions

Algorithm

1. Start
2. Write a source file containing functions: `add`, `sub`, `mul`, `divide`.
3. Write a header file declaring these functions.
4. Compile the source file to an object file.
5. Create a static library (e.g., `libarith.a`) from the object file.
6. Stop.

PSUEDOCODE:

```
MODULE Arithmetic

FUNCTION add(a, b):
    RETURN a + b
END FUNCTION

FUNCTION sub(a, b):
    RETURN a - b
END FUNCTION

FUNCTION mul(a, b):
    RETURN a * b
END FUNCTION

FUNCTION divide(a, b):
    IF b != 0 THEN
        RETURN a / b
    ELSE
        RETURN 0
    ENDIF
END FUNCTION

END MODULE
```


INPUT:

```
1 // use_static.c
2 #include <stdio.h>
3 #include "arith.h"
4 int main(void) {
5     int a = 10, b = 5;
6     printf("add = %d\n", add(a, b));
7     printf("sub = %d\n", sub(a, b));
8     printf("mul = %d\n", mul(a, b));
9     printf("div = %.2f\n", divide(a, b));
10    return 0;
11 }
12
```

OUTPUT:

```
Enter two integers: 2 3
```

```
ADD  = 5
```

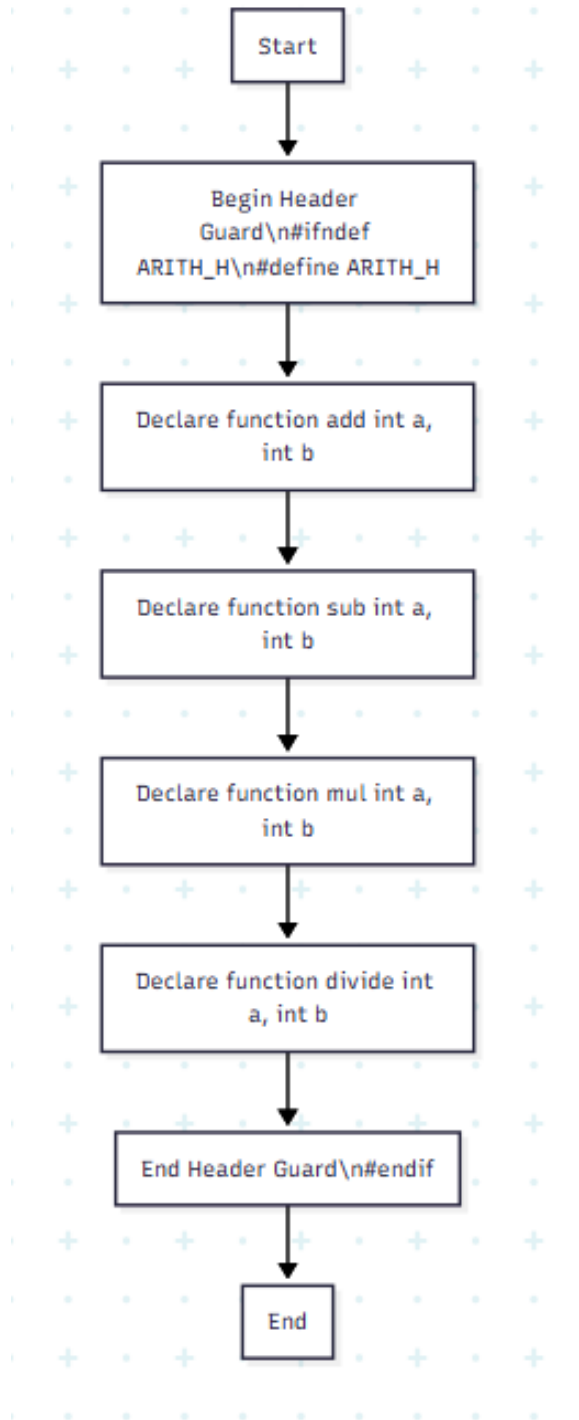
```
SUB  = -1
```

```
MUL  = 6
```

```
DIV  = 0.67
```

```
=== Code Execution Successful ===
```

FLOWCHART:



14.2 – Program to use static library in another program

Algorithm

1. Start
2. Include the arithmetic header file.
3. In `main`, declare two integers `a` and `b`.
4. Assign or read values into `a` and `b`.
5. Call `add(a,b)`, `sub(a,b)`, `mul(a,b)`, `divide(a,b)` – all coming from the static library.
6. Print the results of each function.
7. Stop.

PSUEDOCODE:

```
START

INCLUDE Arithmetic module

DECLARE a, b
SET a = 10
SET b = 5

SET s = add(a, b)
SET d = sub(a, b)
SET p = mul(a, b)
SET q = divide(a, b)

PRINT "add =", s
PRINT "sub =", d
PRINT "mul =", p
PRINT "div =", q

END
```

INPUT:

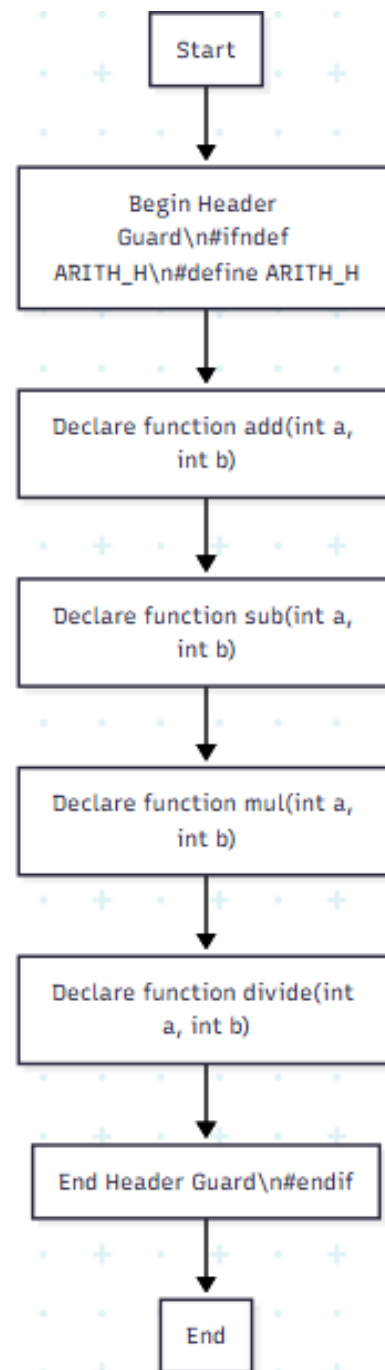
```
1 // use_static.c
2 #include <stdio.h>
3 #include "arith.h"
4 int main(void) {
5     int a = 10, b = 5;
6     printf("add = %d\n", add(a, b));
7     printf("sub = %d\n", sub(a, b));
8     printf("mul = %d\n", mul(a, b));
9     printf("div = %.2f\n", divide(a, b));
10
11     return 0;
12 }
```

OUTPUT:

```
Enter two integers: 8 9
ADD  = 17
SUB  = -1
MUL  = 72
DIV  = 0.89

=== Code Execution Successful ===
```

FLOWCHART:



Experiment 15: Shared Library in C

15.1 – Create shared library

Algorithm

1. Start
2. Use the same arithmetic functions (add, sub, mul, divide) in a source file.
3. Compile the source file as position-independent code.
4. Create a shared library (e.g., `libarith.so`) from the object file.
5. Stop.

PSUEDOCODE:

```
MODULE Arithmetic // to be built as shared library

FUNCTION add(a, b): RETURN a + b
FUNCTION sub(a, b): RETURN a - b
FUNCTION mul(a, b): RETURN a * b
FUNCTION divide(a, b):
    IF b != 0 THEN RETURN a / b
    ELSE RETURN 0
END MODULE
```

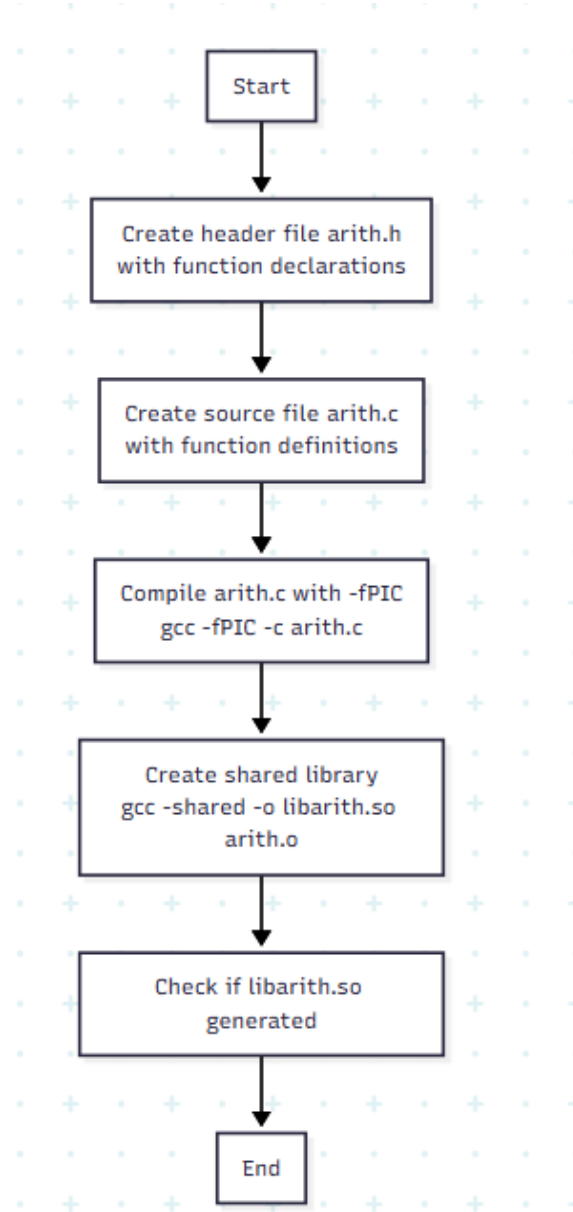
INPUT:

```
bash
```

```
gcc -fPIC -c arith.c          # position independent code
```

```
gcc -shared -o libarith.so arith.o
```

FLOWCHART:



15.2 – Program to use shared library in another program

Algorithm

1. Start
2. Include the arithmetic header file.
3. In `main`, declare two integers `a`, `b`.
4. Read or assign values to `a`, `b`.
5. Call the arithmetic functions from the shared library.
6. Print the results.
7. Stop.

PSUEDOCODE:

```
START

INCLUDE Arithmetic module (from shared library)

DECLARE a, b
SET a = 12
SET b = 4

SET s = add(a, b)
SET d = sub(a, b)
SET p = mul(a, b)
SET q = divide(a, b)

PRINT "add =", s
PRINT "sub =", d
PRINT "mul =", p
PRINT "div =", q

END
```

INPUT:

```
1 // use_shared.c
2 #include <stdio.h>
3 #include "arith.h"
4 int main(void) {
5     int a = 12, b = 4;
6     printf("add = %d\n", add(a, b));
7     printf("sub = %d\n", sub(a, b));
8     printf("mul = %d\n", mul(a, b));
9     printf("div = %.2f\n", divide(a, b));
10    return 0;
11 }
12
```

OUTPUT:

```
Enter two integers: 4 5
```

```
ADD   = 9
```

```
SUB   = -1
```

```
MUL   = 20
```

```
DIV   = 0.80
```

```
=== Code Execution Successful ===
```


FLOWCHART:

